

《人工智能导论》 实验指导书

郑州大学计算机与人工智能学院

2026年

本实验指导书是《人工智能导论》课程的实验指导书，目的是要求学生通过上机实践，能够完成基础的人工智能实验，同时能更深层次的掌握和运用人工智能学科知识，包括熟悉和掌握启发式搜索的定义、估价函数和算法过程，学会使用归结原理完成机器自动定理证明，学会使用决策树完成分类任务，熟悉和掌握遗传算法的基本过程，加深对神经网络、回归等概念的理解，熟悉和掌握循环神经网络的模型架构和工作原理。

本实验设计了八个综合性实验，实验内容包括利用遗传算法求解TSP问题、根据决策树算法进行分类学习（西瓜分类和辅助诊断）、使用全连接网络实现房价预测、实现商品评论情感分析（基于大语言模型及循环神经网络）、A*算法解决八数码问题、使用归结原理进行定理证明等。针对学生的实际情况，实验内容和实验题目的设计做到难易适中，各个实验在实验要求上分成不同的层次，力争让学生经过一定的努力，都能够完成相应的题目，从而激发学生的学习兴趣和积极性。

本指导书主要用于《人工智能》课程的实验指导，可供计算机科学与技术、软件工程以及计算机学科其他专业选用。

目 录

实验一 利用遗传算法求解TSP问题	1
一、实验目的	1
二、实验内容	1
三、实验环境	2
四、实验原理	2
五、实验步骤	3
六、实验要求	7
实验二 决策树实验（疾病分类）	8
一、实验目的	8
二、实验内容	8
三、实验环境	8
四、实验原理	9
五、实验步骤	11
六、实验要求	17
实验三 基于全连接网络实现房价预测	19
一、实验目的	19
二、实验内容	19
三、实验环境	19
四、实验原理	19
五、实验步骤	20
六、实验要求	24

实验四 基于Llama 3实现商品评论情感分析	25
一、实验目的	25
二、实验内容	25
三、实验环境	25
四、实验原理	26
五、实验步骤	27
六、实验要求	32
实验五 A*算法之八数码问题	33
一、实验目的	33
二、实验内容	33
三、实验环境	33
四、实验原理	33
五、实验步骤	34
六、实验要求	38
实验六 归结原理实验	40
一、实验目的	40
二、实验内容	40
三、实验环境	40
四、实验原理	40
五、实验步骤	40
六、实验要求	46
实验七 决策树实验(西瓜分类) (*)	48
一、实验目的	48

二、实验内容	48
三、实验环境	48
四、实验原理	48
五、实验步骤	50
六、实验要求	53
实验八 基于循环神经网络实现情感分类（*）	54
一、实验目的	54
二、实验内容	54
三、实验环境	54
四、实验原理	54
五、实验步骤	55
六、实验要求	60
附录A：实验报告封皮格式	61
附录B：实验报告格式	62

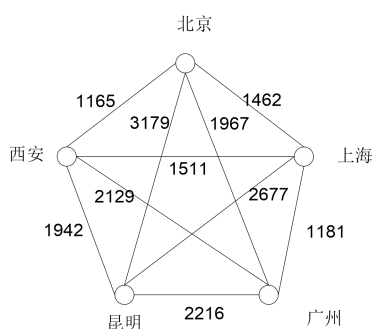
实验一 利用遗传算法求解TSP问题

一、实验目的

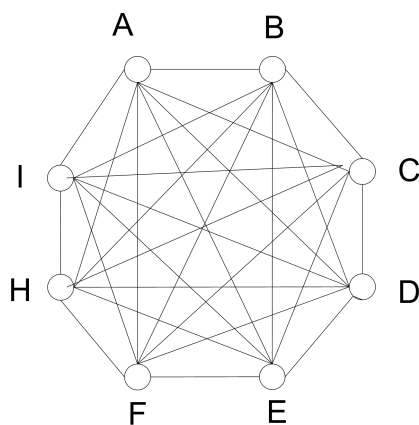
熟悉和掌握遗传算法的基本过程，掌握选择、变异和交叉等重要操作算子的设计，对具体的问题学习设计种群中个体的编码，掌握适应度函数的设计，加深对遗传算法的理解，并利用遗传算法解决TSP问题。

二、实验内容

1、使用遗传算法求下图中从北京出发经过其他四个城市之后回到北京的最短路径，两个城市之间的距离如图所示。



2、使用遗传算法求下图中从A出发经过其他七个城市之后回到A的最短路径。



其中任意两个城市之间的距离如下表所示：

	A	B	C	D	E	F	H	I
A	0							
B	49	0						
C	25	26	0					
D	19	48	26	0				
E	63	65	21	45	0			

F	74	36	24	44	47	0		
H	26	42	78	57	48	47	0	
I	39	55	49	62	54	65	47	0

以上两题的矩阵表示如下：

```

1. # 实验一
   city_dist_mat = np.zeros([city_num, city_num])
2. city_dist_mat[0][1] = city_dist_mat[1][0] = 1165
3. city_dist_mat[0][2] = city_dist_mat[2][0] = 1462
4. city_dist_mat[0][3] = city_dist_mat[3][0] = 3179
5. city_dist_mat[0][4] = city_dist_mat[4][0] = 1967
6. city_dist_mat[1][2] = city_dist_mat[2][1] = 1511
7. city_dist_mat[1][3] = city_dist_mat[3][1] = 1942
8. city_dist_mat[1][4] = city_dist_mat[4][1] = 2129
9. city_dist_mat[2][3] = city_dist_mat[3][2] = 2677
10. city_dist_mat[2][4] = city_dist_mat[4][2] = 1181
11. city_dist_mat[3][4] = city_dist_mat[4][3] = 2216

```

```

1. # 实验二
2. city_dist_mat = np.array([
3.     [0, 49, 25, 19, 63, 74, 26, 39],
4.     [49, 0, 26, 48, 65, 36, 42, 55],
5.     [25, 26, 0, 26, 21, 24, 78, 49],
6.     [19, 48, 26, 0, 45, 44, 57, 62],
7.     [63, 65, 21, 45, 0, 47, 48, 54],
8.     [74, 36, 24, 44, 47, 0, 47, 65],
9.     [26, 42, 78, 57, 48, 47, 0, 47],
10.    [39, 55, 49, 62, 54, 65, 47, 0]
11. ])
12.

```

三、实验环境

推荐使用Python语言，编译器推荐使用PyCharm 2021.2.1(Professional Edition)，环境如下：

python=3.8.0

prettytable=3.1.1

四、实验原理

遗传算法（Genetic Algorithm, GA）是模拟达尔文生物进化论的自然选择和遗传学机理的生物进化过程的计算模型，是一种通过模拟自然进化过程搜索最优解的方法。该算法以一种群体中的所有个体为对象，并利用随机化技术指导对一个被编码的参数空间进行高效搜索。其中，选择、交叉和变异构成了遗传算法的遗传操

作；参数编码、初始群体的设定、适应度函数的设计、遗传操作设计、控制参数设定五个要素组成了遗传算法的核心内容。

五、实验步骤

步骤一 针对TSP问题，确定编码。

可采用十进制编码法，对城市进行编号，每个城市分别用1到n之间不同的整数表示，n个整数的一个排列就代表了旅行商问题。如对于五城市旅行商问题，用1到5分别代表城市北京、上海、广州、昆明、西安，其中一个可能解为：

1 5 2 3 4 1（北京-西安-上海-广州-昆明-北京）

定义个体类，每个个体类中包括基因（城市路线）和适应度，核心代码如下：

```
1. class Individual:
2.     def __init__(self, genes=None):
3.         if genes == None:
4.             genes = [i for i in range(city_num)]
5.             random.shuffle(genes)
6.             self.genes = genes
7.             self.fitness = self.evaluate_fitness()
8.             # 计算个体的适应度
9.     def evaluate_fitness(self):
               .....
```

步骤二 针对TSP问题，适应度函数可定义为：

$$f(s) = \frac{1}{\sum_{i=1}^n d(c_i, c_{i+1})} \quad (2)$$

其中d(c_i, c_{i+1})表示相邻城市之间的距离。

步骤三 针对TSP问题，确定交叉规则。对于采用整数编码表示的染色体，可以有以下交叉规则：

（1）顺序交叉法（Order Crossover, OX）

在两个父代染色体中随机选择起始和结束位置，将父代染色体1该区域内的基因复制到子代1相同位置上，再在父代染色体2上将子代1中缺少的基因按照顺序填入。另一个子代以类似方式得到。

起始位置为1，结束位置为5

父代1: 1 2 3 4 5 6 7 8

父代2: 5 2 1 7 3 8 6 4

子代1: 1 2 3 4 5 6 7 8

子代2: 4 2 1 7 3 8 5 6

(2) 基于顺序的交叉法 (Order-Based Crossover, OBX)

在两个父代染色体中随机选择几个位置，位置可以不连续，先在父代染色体2中找到父代染色体1被选中基因的位置，再用父代染色体2中其余的基因生成子代，并保证位置对应，将父代染色体1中被选择的基因按顺序放入子代剩余位置中。另一个子代以类似方式得到。

父代1: 1 2 3 4 5

父代2: 5 3 2 4 1

所选位置: * * *

从父代1中选出相应位数，删除父代2中同样数子，按父代1中数子顺序填入父代2

子代1: 4 3 2 5 1

从父代2中选出相应位数，删除父代1中同样数子，按父代2中数子顺序填入父代2

子代2: 4 2 3 1 5

(3) 基于位置的交叉法 (Position-based Crossover, PBX)

对于选定的父代染色体父代1和父代2，首先随机产生一组位置。对于这些位置上的基因，子代1从父代2中直接得到，子代1其他位置的基因，按照顺序从父代1中选取那些不相重的基因。子代2也进行类似的处理。PBX与OBX相比，生成子代的“基础”基因来源不同，PBX来自被选中基因，OBX来自剩余基因。

父代1: 1 2 3 4 5

父代2: 5 2 4 1 3

所选位置* * *

子代1: 1 5 3 4 2

子代2: 5 2 4 1 3

步骤四 确定变异规则，以下三种变异规则可任选一种。

(1) 基于位置的变异:

该方法随机地产生两个变异位，然后将第二个变异位上的基因移动到第一个

变异位之前。如假定变异位为2和5，对整数编码的TSP问题，变异前后的变化为：

变异前：1 2 3 4 5

变异后：1 5 2 3 4

(2) 基于次序的变异：

该方法随机地产生两个变异位，然后交换这两个变异位上的基因。如假定变异位为2和5，对整数编码的TSP问题，变异前后的变化为：

变异前：1 2 3 4 5

变异后：1 5 3 4 2

(3) 打乱变异：

该方法随机地选取染色体上的一段，然后打乱在该段内的基因次序。如随机选择段的染色体为第二到第五位，对整数编码的TSP问题，其中一种变异的结果为：

变异前：1 2 3 4 5

变异后：1 3 5 4 2

(4) 翻转切片变异

该方法随机产生两个变异位，作为起始位置和结束位置，将两位置之间的基因翻转，变异前后的变化为：

变异前：1 2 3 4 5

变异后：1 4 3 2 5

步骤五 根据种群选择算法

1. 锦标赛算法 (Tournament Selection)

锦标赛方法选择策略每次从种群中取出一定数量个体，然后选择其中最好的一个进入子代种群。重复该操作，直到新的种群规模达到原来的种群规模。具体的操作步骤如下：

(1) 确定每次选择的个体数量(本文以占种群中个体个数的百分比表示)。

(2) 从种群中随机选择个体(每个个体入选概率相同) 构成组，根据每个个体的适应度值，选择其中适应度值最好的个体进入子代种群。

(3) 重复步骤2，得到的个体构成新一代种群。

2. 轮盘赌选择算法 (Roulette Wheel Selection)

它模拟博彩游戏的轮盘赌。一个轮盘被划分为N个扇形表示种群的一个染色体，而每个扇形的面积与它所表示的染色体的适应值成正比，为了选择种群中的个体，

设想有一个指针指向轮盘，转动轮盘，当轮盘停止后，指针所之的染色体被选择。因为一个染色体的适应值越大表示该染色体的扇形面积就越大，因此它被选择的可能性也就越大。

步骤六 根据选定的编码、遗传操作实现基本遗传算法，编写代码，调试程序。输出结果。

定义Ga类，用于执行遗传算法，核心代码如下所示。

```

1.  class Ga:
2.      # input_ 为城市间的距离矩阵
3.      def __init__(self, input_):
4.          global city_dist_mat
5.          city_dist_mat = input_
6.          # 当代的最佳个体（距离最短路径）
7.          self.best = None
8.          # 种群
9.          self.individual_list = []
10.         # 每一代的最佳个体
11.         self.result_list = []
12.         # 每一代个体对应的最佳适应度
13.         self.fitness_list = []
14.
15.         # 交叉，选择合适算法实现
16.         def cross(self):
17.             .....
18.         # 变异，选择合适算法实现
19.         def mutate(self, new_gene):
20.             .....
21.         # 选择，选择合适算法实现
22.         def select(self):
23.             .....
24.
25.         # 更新种群
26.         def next_gen(self):
27.             # 交叉，产生子代
28.             new_gene = self.cross()
29.             # 变异
30.             self.mutate(new_gene)
31.             # 选择
32.             self.select()
33.             # 获得这一代的最佳个体
34.             for individual in self.individual_list:
35.                 if individual.fitness > self.best.fitness:
36.                     self.best = individual
37.
38.         def train(self):
39.             # 随机出初代种群
40.             self.individual_list = [Individual() for _ in range(individual_num)]

```

```
41.     self.best = self.individual_list[0]
42.     # 迭代
43.     for i in range(gen_num):
44.         # 从当代种群中交叉、变异、选择出适应度最佳的个体，获得子代产生新的种群
45.         self.next_gen()
46.         # 连接首位
47.         result = copy.deepcopy(self.best.genes)
48.         result.append(result[0])
49.         self.result_list.append(result)
50.         self.fitness_list.append(self.best.fitness)
51.     return self.result_list, self.fitness_list
```

六、实验要求

1. 按照实验步骤选择合适的算法编码完成实验内容中的两道题目，最终得到距离最短的城市列表。
2. 选择合适的算法实现交叉、变异、选择，根据核心代码完成实验。

实验二 决策树实验（疾病分类）

一、实验目的

1. 掌握决策树分类方法，通过理论研习与实操，深入理解决策树基于树形结构的分类逻辑，包括节点划分及决策流程，明晰其的分类标准。
2. 掌握ID3、C4.5和CART这三种经典算法决策树原理与实现机制，并选用其中一个算法进行实验，能熟练构建决策树。
3. 学会进行数据预处理，掌握数据集的预处理方法，如清洗缺失值与异常值、转换属性值为唯一标签、合理划分训练集与测试集，从而保障决策树算法高效运行，增强应对实际数据问题的本领。

二、实验内容

本次实验需运用决策树算法完成对糖尿病数据集的分类任务。糖尿病数据集共768条样本，未患病的样本个数为500个，患病的样本个数为268个。包含Pregnancies（怀孕次数）、Glucose（血糖）、BloodPressure（血压）、SkinThickness（皮肤厚度）、Insulin（胰岛素）、BMI（身体质量指数）、DiabetesPedigreeFunction（糖尿病遗传功能）、Age（年龄）这些属性，通过这些属性预测 Outcome（是否患糖尿病，1 表示是，0 表示否），部分样例如下表。

编号	Preg	Gluc	BP	SkinT	Ins	BMI	DPF	Age	Outcome
1	6	148	72	35	0	33.6	0.627	50	1
2	1	89	66	23	94	28.1	0.167	21	0
3	8	183	64	0	0	23.3	0.672	32	1

*Preg: 怀孕次数; Gluc: 血糖; BP: 血压; SkinT: 皮肤厚度; Ins: 胰岛素; DPF: 糖尿病遗传功能

三、实验环境

推荐使用python语言，编译器推荐pycharm 2021.2.1，环境如下：

Python	matplotlib	pandas	scikit-learn
Python 3.7.0	3.10.1	2.2.3	1.6.1

*上述库版本供参考，实际环境要求并不严苛。可依据具体实现，自主决定是否安装各库。若需可视化，graphviz 可作为 matplotlib 的替代工具；pandas 主要用于数据读取与处理；scikit - learn 则可实现数据集分割。

四、实验原理

决策树是一种描述对实例进行分类的树形结构。决策树由点和有向边组成。节点有两种类型：内部节点和叶节点。内部节点表示一种特征或者属性，叶节点表示一个分类。构建决策树时通常采用自上而下的方法，在每一步选择一个最好的属性来分裂。每次都找不同的切分点，将样本空间逐渐进行细分，最后把属于同一类的空间进行合并，就形成了决策边界，树的层次越深，决策边界的切分就越细，区分越准确，同时也越有可能产生过拟合。

决策树的建立开始，构建根节点，将所有训练数据放在根节点，选择一个最优特征，按照这一特征的取值将训练数据分割为子集，使各个子集有一个当前条件下最好的分类。如果这些子集能被基本正确分类，那么构造叶节点，将对应子集集中到叶节点。如果有子集不能被正确分类，那么就这些子集选择新的最优特征，继续对其进行分割，构建相应的节点。递归进行上述的操作，直到所有训练数据子集均能被正确分类。

信息熵(entropy) 熵是接收的每条消息中包含的信息的平均量，熵是对不确定性的测量。但是在信息世界，熵越高，则能传输越多的信息，熵越低，则意味着传输的信息越少。简单来说，熵表示随机变量的不确定性。

条件熵(conditional entropy) 在一个条件下，随机变量的不确定性。

信息增益(information gain) 信息增益=熵-条件熵。表示在一个条件下，信息不确定性减少的程度。

信息增益率(information gain ratio) 节点信息增益与节点分裂信息度量的比值。

ID3算法 ID3算法的核心是在决策树的各个结点上应用信息增益准则进行特征选择。

C4.5算法 相比ID3算法，C4.5算法更换了特征选择的标准，使用信息增益比进行特征选择。不直接选择增益率最大的候选划分属性，候选划分属性中找出信息增益高于平均水平的属性（这样保证了大部分好的的特征），再从中选择增益率最高的（又保证了不会出现编号特征这种极端的情况）对于连续值属性来说，可取值数目不再有限，因此可以采用离散化技术（如二分法）进行处理。将属性值从小到大排序，然后选择中间值作为分割点，数值比它小的点被划分到左子树，数值不小

于它的点被分到叉子树，计算分割的信息增益率，选择信息增益率最大的属性值进行分割。

CART算法 CART中用于选择变量的不纯度度量是Gini指数；如果目标变量是标称的，并且是具有两个以上的类别，则CART可能考虑将目标类别合并成两个超类别（双化）；如果目标变量是连续的，则CART算法找出一组基于树的回归方程来预测目标变量。CART是一棵二叉树，采用二元切分法，每次把数据切成两份，分别进入左子树、右子树。而且每个非叶子节点都有两个孩子，所以CART的叶子节点比非叶子多1。相比ID3和C4.5，CART应用要多一些，既可以用于分类也可以用于回归。CART分类时，使用基尼指数（Gini）来选择最好的数据分割的特征，gini描述的是纯度，与信息熵的含义相似。CART中每一次迭代都会降低GINI系数。

剪枝 剪枝为了避免决策树“过拟合”样本，决策树的剪枝策略最基本的有两种：预剪枝（pre-pruning）和后剪枝（post-pruning）。

预剪枝（pre-pruning）：预剪枝就是在构造决策树的过程中，先对每个结点在划分前进行估计，若果当前结点的划分不能带来决策树模型泛华性能的提升，则不对当前结点进行划分并且将当前结点标记为叶结点。

后剪枝（post-pruning）：后剪枝就是先把整颗决策树构造完毕，然后自底向上的对非叶结点进行考察，若将该结点对应的子树换为叶结点能够带来泛华性能的提升，则把该子树替换为叶结点。

代价复杂度剪枝（Cost-Complexity Pruning，简称CCP）是决策树中一种常用的后剪枝算法。CCP算法通过在模型复杂度和预测误差之间寻找平衡点，生成一系列嵌套的简化决策树。CCP引入了一个复杂度参数 α ，用于权衡剪枝子树的误差增加与复杂度降低之间的关系，树的层次越浅，模型越简单，泛化能力越强，同时也可能导致欠拟合。

CCP算法的执行过程是，首先构建一棵完全生长的决策树，然后计算每个内部节点的代价复杂度参数 α 。这个参数衡量了将该节点剪枝后增加的误差与减少的复杂度比值。之后，算法从 α 值最小的节点开始剪枝，每次剪枝后都会形成一棵新的子树。剪枝过程自底向上进行，直到最后只剩下根节点，形成了一个从最复杂到最简单的决策树序列。

代价复杂度参数 α 定义为将内部节点剪成叶节点后带来的误差增加与复杂度减少的比值。对于内部节点 t ，其 α 值计算为： $\alpha(t) = [R(t) - R(T_t)] / [|T_t|$

- 1], 其中 $R(t)$ 是将 t 作为叶节点的误差, $R(T_t)$ 是以 t 为根的子树误差, $|T_t|$ 是子树的叶节点数。

剪枝树序列 是通过不同 α 值产生的一系列嵌套决策树。随着 α 值增大, 树的复杂度逐渐降低, 从完全树一直简化到只有根节点的单节点树。

CCP算法通过最小化代价复杂度度量 $R_\alpha(T) = R(T) + \alpha |T|$ 来选择最优子树。这个度量将预测误差 $R(T)$ 和树复杂度 $|T|$ 结合起来, α 控制了两者之间的权衡关系。

CCP算法首先构建完全树 T_0 ; 然后计算每个内部节点的 α 值, 选择 α 最小的节点进行剪枝, 生成 T_1 ; 重复计算和剪枝过程, 得到树序列 $T_0 > T_1 > \dots > T_n$; 最后使用独立验证集选择序列中的最优树, 或采用1-SE规则选择在误差可接受范围内最简单的树。

相比错误率降低剪枝(REP), CCP考虑了树的复杂度, 能更有效地控制过拟合; 相比悲观错误剪枝(PEP), CCP基于实际计算而非估计, 结果更准确; 相比MDL剪枝, CCP的计算更简单高效。CCP生成的是一系列候选树而非单一结果, 为模型选择提供了更大灵活性。CCP被广泛应用于CART决策树中, 在scikit-learn等机器学习库中通过ccp_alpha参数控制剪枝强度。

五、实验步骤

以糖尿病的CART决策树算法为例:

步骤一 对糖尿病数据集中的连续型特征进行分箱（离散化）处理生成对应的分类变量。先尝试用pandas库中的qcut等频分箱, 它按分位数将数据划分到指定区间, 保证各区间样本数均衡。若因数据重复值多导致等频分箱失败, 就用cut等宽分箱, 以固定宽度分割区间。分箱默认类别标签是['0', '1', '2']。分箱能把连续变量转为离散变量, 便于模型处理, 同时增强结果的可解释性, 让模型在不同数据集上表现更优。

#	A	B	C	D	E	F	G	H	I
1	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
2	6	148	72	35	0	33.6	0.627	50	1
3	1	85	66	29	0	26.6	0.351	31	0
4	8	183	64	0	0	23.3	0.672	32	1
5	1	89	66	23	94	28.1	0.167	21	0
6	0	137	40	35	168	43.1	2.288	33	1
7	5	116	74	0	0	25.6	0.201	30	0
8	3	78	50	32	88	31	0.248	26	1
9	10	115	0	0	0	35.3	0.134	29	0
10	2	197	70	45	543	30.5	0.158	53	1
11	8	125	96	0	0	0	0.232	54	1
12	4	110	92	0	0	37.6	0.191	30	0
13	10	168	74	0	0	38	0.537	34	1
14	10	139	80	0	0	27.1	1.441	57	0
15	1	189	60	23	846	30.1	0.398	59	1
16	5	166	72	19	175	25.8	0.587	51	1
17	7	100	0	0	0	30	0.484	32	1

图 1 原始数据

```
1. import pandas as pd
```

```

2.
3. def binning_classify(file_path, bins=3, labels=['0', '1', '2']):
4.     df = pd.read_excel(file_path)
5.     for col in df.columns[:-1]: # 遍历除最后一列外的所有列
6.         try:
7.             # 直接覆盖原列: 将分箱结果赋值给原列名 `col`
8.             df[col] = pd.qcut(df[col], q=bins, labels=labels)
9.         except:
10.            df[col] = pd.cut(df[col], bins=bins, labels=labels)
11.     return df
12.
13. input_file = 'Diabets.xlsx'
14. result_df = binning_classify(input_file)
15. result_df.to_excel('classified_output.xlsx', index=False)

```

	A	B	C	D	E	F	G	H	I
1	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	Pedigree	Age	Outcome
2	2	2	1	2	0	1	2	2	1
3	0	0	0	1	0	0	1	1	0
4	2	2	0	0	0	0	2	1	1
5	0	0	0	1	0	0	0	0	0
6	0	2	0	2	0	2	2	1	1
7	1	1	1	0	0	0	0	1	0
8	1	0	0	2	0	1	0	1	1
9	2	1	0	0	0	2	0	1	0
10	0	2	1	2	1	1	0	2	1
11	2	1	2	0	0	0	0	2	1
12	1	1	2	0	0	2	0	1	0
13	2	2	1	0	0	2	2	1	1
14	2	2	2	0	0	0	2	2	0
15	0	0	0	1	0	1	1	0	1

图 2 分箱后的数据

步骤二 将数据集按照目标列进行分层抽样，分割为训练集、验证集和测试集，比例为 7:2:1。

```

1. from sklearn.model_selection import train_test_split
2. def stratified_split(data, target_column=None, ratios=[0.7, 0.2, 0.1], seed=42):
3.     """
4.     按目标列进行分层抽样分割
5.     :param data: 输入的 DataFrame
6.     :param target_column: 目标列名（默认为最后一列）
7.     :param ratios: 训练集/验证集/测试集比例
8.     :param seed: 随机种子
9.     :return: 分割后的三个 DataFrame
10.    """
11.    if target_column is None:
12.        target_column = data.columns[-1] # 默认使用最后一列
13.
14.    # 验证比例有效性，允许浮点数精度误差
15.    if not abs(sum(ratios) - 1.0) < 1e-9:
16.        raise ValueError("ratios must sum to 1.0")
17.
18.    # 第一步分割训练集和临时集
19.    train_ratio = ratios[0]

```

```
20. train, temp = train_test_split(
21.     data,
22.     train_size=train_ratio,
23.     stratify=data[target_column],
24.     random_state=seed
25. )
26.
27. # 第二步分割验证集和测试集
28. val_ratio = ratios[1] / (ratios[1] + ratios[2])
29. validation, test = train_test_split(
30.     temp,
31.     train_size=val_ratio,
32.     stratify=temp[target_column],
33.     random_state=seed
34. )
35.
36. return train, validation, test
37.
38. df = pd.read_excel('classified_output.xlsx')
39.
40. ratios = [0.7, 0.2, 0.1]
41.
42. # 执行分层抽样
43. train_data, validation_data, test_data = stratified_split(
44.     data=df,
45.     target_column=df.columns[-1],
46.     ratios=ratios,
47.     seed=42
48. )
49.
```

```
Train set distribution:
Outcome
0    0.651769
1    0.348231
Name: proportion, dtype: float64

Validation set distribution:
Outcome
0    0.649351
1    0.350649
Name: proportion, dtype: float64

Test set distribution:
Outcome
0    0.649351
1    0.350649
Name: proportion, dtype: float64
```

图 3 集合划分

步骤三 使用CART算法来构造决策树，评估并进行可视化。但实际上数据集较大时未剪枝的决策树的可视化效果不好，在此进行了预剪枝。核心代码如下：

```

1. train_data_list = train_data.values.tolist()
2. validation_data_list = validation_data.values.tolist()
3. test_data_list = test_data.values.tolist()
4.
5. max_depth = 4
6. min_size = 20
7. tree = build_tree(train_data_list, max_depth, min_size)
8. predictions = []
9. for row in test_data_list:
10.     prediction = predict(tree, row)
11.     if prediction is not None:
12.         predictions.append(prediction)
13.
14. if len(predictions) > 0:
15.     correct = 0
16.     for i in range(len(predictions)):
17.         if test_data_list[i][-1] == predictions[i]:
18.             correct += 1
19.     accuracy = correct / float(len(predictions)) * 100.0
20.     print(f"决策树准确率: {accuracy:.2f}%")
21. else:
22.     print("No valid predictions were made.")
23. labels = ["Pregnancies", "Glucose", "BloodPressure", "SkinThickness", "Insulin", "BMI",
24.           "DiabetesPedigreeFunction", "Age", "Outcome"]
25. createPlot(tree, labels)

```

步骤四 对CART算法构造的决策树进行代价复杂度剪枝。剪枝方法多种多样，主要目的是防止过拟合，也可以使用其他相对简单的剪枝算法。使用自己定义的calculate_alpha_values函数，基于验证集计算每个内部节点的 alpha 值。alpha 值反映了剪枝后误差增加与复杂度减少的比率。核心思想如下：

首先计算节点在数据上的误差率。对于给定的节点和数据，如果数据为空，误差率为 0；若节点是内部节点（字典形式），用它预测数据的标签，否则将节点值作为所有样本的预测，然后比较预测结果与实际标签，计算错误比例。接着，计算树的复杂度，如果节点是内部节点，递归求解左子树和右子树的复杂度之和，若为叶节点，复杂度为 1。之后，获取树的内部节点。从根节点开始，用栈遍历树，当节点是字典且包含“索引”和“值”属性时，记录为内部节点，并将字典形式的左右子树加入栈，最终返回所有内部节点。

然后，确定到达特定节点的样本，遍历数据，筛选出能通过节点条件到达目标节点的样本。基于此，计算 alpha 值：对于每个内部节点，找到其验证集样本，计算当前子树的误差和复杂度，再假设将子树替换为叶节点，计算新误差，用（叶

节点误差 - 当前误差) / (复杂度 - 1) 得到 α 值并记录。剪枝过程则是在树中定位目标节点。若当前节点就是要剪掉的节点，将其替换为叶节点值，否则递归处理左右子树，更新并返回树结构。

剪枝的主流程是先计算所有内部节点的 α 值，若为空则返回原树；否则，初始化候选树列表（包含原树， α 为 0），按 α 值排序节点，逐一剪掉节点生成新树并加入候选列表，最后从这些树中选出在验证集上误差最小的树。整个过程以主程序调用结束。用初始树和验证集运行剪枝流程，得到最终的剪枝树。这种方法通过 α 值平衡误差和复杂度，确保剪枝后树在验证集上表现最优。

步骤五 合并叶节点。以下是一张利用scikit-learn实现的无预剪枝、使用CCP算法剪枝的CART决策树可视化图像。可以看到红框圈出的部分存在冗余分支。既然最终分类结果一样，那么在“Age <= 0.5”（Age离散化成了0, 1, 2三个标签）这个条件分支下，没必要再细分出这两个子节点进行判断，增加了模型的复杂度却没有带来分类结果上的差异，属于无效的结构构建，浪费计算资源和存储空间。

同理，我们自己实现的CCP算法也无法去除这种冗余分支，为了使剪枝得到的最终结果更简洁，我们实现了以下的合并冗余节点的代码，可以剪枝后使用。

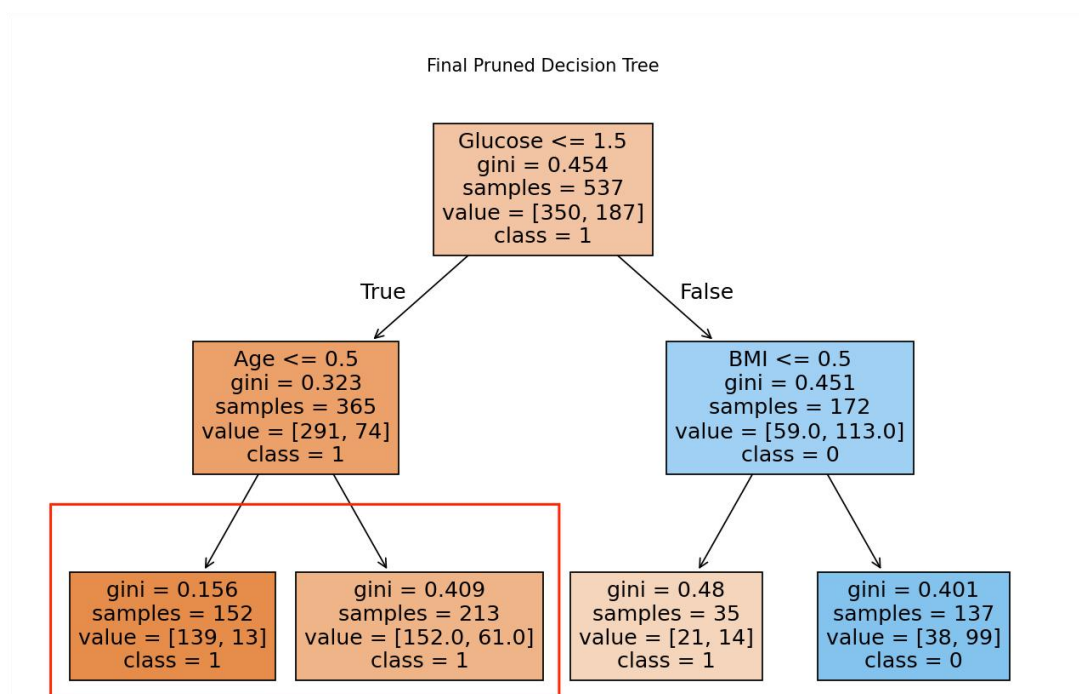


图 4 冗余结点示意

*此图是采用scikit-learn库实现的ccp剪枝后的图像，可以看到左下角的两个叶子结点冗余。

```
1. def merge_nodes(node):
```

```

2.     """
3.     合并预测值相同的相邻叶节点
4.     """
5.     if isinstance(node, dict):
6.         # 递归处理左右子树
7.         new_left = merge_nodes(node['left'])
8.         new_right = merge_nodes(node['right'])
9.
10.        # 检查是否都是叶子节点且预测值相同
11.        if isinstance(new_left, int) and isinstance(new_right, int) and new_left == new_right:
12.            return new_left # 合并为单个叶子节点
13.        else:
14.            # 更新子节点引用
15.            node['left'] = new_left
16.            node['right'] = new_right
17.            return node
18.    else:
19.        return node # 叶子节点直接返回
20.
21. pruned_tree = merge_nodes(pruned_tree)

```

步骤六 模型评估和可视化，以下是一个简单的示例，还可增加其他评估指标。

```

1. def evaluate_model(model, test_data, name=""):
2.     """
3.     评估模型在测试集上的性能
4.     """
5.     predictions = []
6.     for row in test_data:
7.         prediction = predict(model, row)
8.         if prediction is not None:
9.             predictions.append(prediction)
10.
11.    if len(predictions) > 0:
12.        actual = [row[-1] for row in test_data]
13.        correct = sum(1 for a, p in zip(actual, predictions) if a == p)
14.        accuracy = correct / len(predictions) * 100.0
15.        print(f"{name}准确率: {accuracy:.2f}%")
16.        return accuracy
17.    else:
18.        print(f"{name}没有有效预测")
19.        return 0.0
20.
21. test_data_list = test_data.values.tolist()
22. print("\n--- 未剪枝的决策树评估 ---")
23. evaluate_model(tree, test_data_list, "未剪枝决策树")
24. print("\n--- 剪枝后的决策树评估 ---")
25. evaluate_model(pruned_tree, test_data_list, "代价复杂度剪枝")

```

可视化决策树，结合特征名称列表labels，可视化未剪枝和剪枝后的决策树，直观观察树结构变化。也可使用别的库实现。

```

1. labels = ["Pregnancies", "Glucose", "BloodPressure", "SkinThickness", "Insulin", "BMI",
2.           "DiabetesPedigreeFunction", "Age", "Outcome"]
3. print("\n--- 可视化未剪枝的决策树 ---")
4. createPlot(tree, labels)
5. print("\n--- 可视化剪枝后的决策树 ---")
6. createPlot(pruned_tree, labels)

```

计算并输出未剪枝决策树和剪枝后决策树的复杂度（叶节点数量），比较剪枝前后的复杂度变化。

```

1. original_complexity = calculate_complexity(tree)
2. pruned_complexity = calculate_complexity(pruned_tree)
3. print(f"\n--- 复杂度比较 ---")
4. print(f"原始树叶节点数量: {original_complexity}")
5. print(f"剪枝后叶节点数量: {pruned_complexity}")
6. print(f"复杂度减少: {original_complexity - pruned_complexity} 个叶节点 ({(original_complexity - pruned_complexity) / original_complexity * 100:.2f}%)")

```

```

--- 未剪枝的决策树评估 ---
未剪枝决策树准确率: 77.92%

--- 可视化未剪枝的决策树 ---

--- 执行代价复杂度剪枝 ---

--- 剪枝后的决策树评估 ---
代价复杂度剪枝准确率: 80.52%

--- 可视化剪枝后的决策树 ---

--- 复杂度比较 ---
原始树叶节点数量: 6
剪枝后叶节点数量: 3
复杂度减少: 3 个叶节点 (50.00%)

```

图 5 参考结果

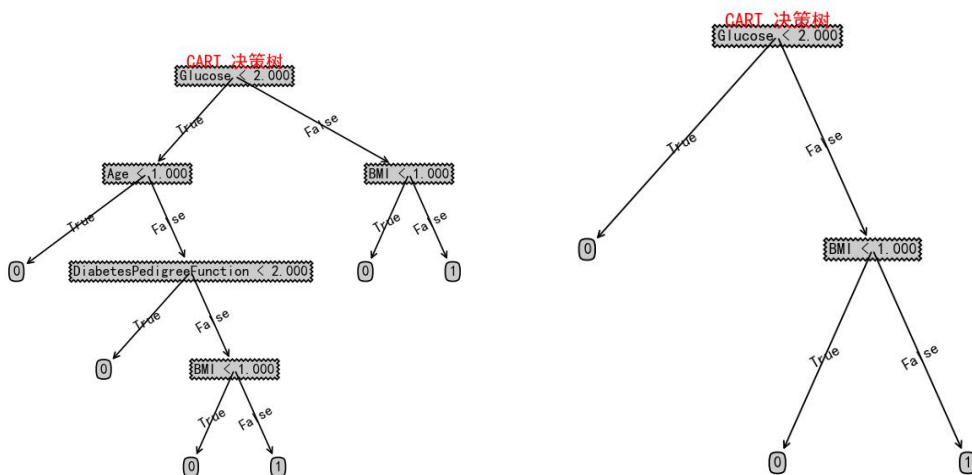


图 6 可视化决策树参考

*左子图为未剪枝时的决策树（但合并了冗余结点），右子图为剪枝后的决策树。为了保证可视化效果采用的结点数不多。还可以进一步优化以上提供的可视化函数，除了展示树的结构，可在可视化图形上添加更多信息，如每个节点的样本数量占比、信息增益或基尼指数的具体数值等。

六、实验要求

1. 数据处理与划分

- (1) 数据集划分：将数据集划分为训练集、验证集和测试集。其中，验证集用于决策树的剪枝操作，测试集用于评估决策树模型的效果。
- (2) 分层抽样：为保证划分的合理性，采用分层抽样方法，使正负样本在训练集、验证集和测试集中的比例大致均衡。
- (3) 连续变量处理：对于连续变量，必须进行离散化处理。

2. 决策树算法实现：

用任意编程语言实现一个决策树算法，可基于信息熵（如 ID3、C4.5）或基尼指数（CART）进行划分选择。

3. 剪枝操作

- (1) 剪枝必要性：剪枝是防止决策树模型过拟合的重要手段，通常需要实现预剪枝或后剪枝中的一种。
- (2) 效果对比：在实验中需对比剪枝前后的效果。一般来说，剪枝后树的节点数会减少，测试集上的表现可能提升、不变或下降，具体情况取决于具体数据和剪枝策略，并非一定会有显著改善。

4. 决策树可视化：依据所实现的算法构造决策树，并将其进行可视化展示。

实验三 基于全连接网络实现房价预测

一、实验目的

加深对神经网络、回归等概念的理解，掌握模型的构建、损失函数的定义、优化器的选择、网络的前向计算、网络的反向传播等关键操作；并且熟悉使用飞桨（Paddle Paddle）深度学习框架来进行以上的操作。

二、实验内容

飞桨内部集成了波士顿房价数据集uci-housing，该数据集共506条数据，每条数据包含14个值，其中前13个值用来描述房屋的各种信息，作为特征信息，最后一个值为该类房屋价格中位数，即要回归预测的值。

本实验要求使用飞桨搭建单层的全连接网络，并利用上述数据集训练该模型；训练结束后保存模型，存储模型的参数；评估网络时，从参数文件中加载模型参数，再评估模型。

三、实验环境

1. 建议使用百度提供的AI Studio平台做下面两个实验，该平台提供了免费的算力支持，并且已经集成了我们要用的paddle等库。

<https://aistudio.baidu.com/aistudio/index>

进入网页后—》点击项目—》创建项目—》Notebook—》选paddle 2.0.2版本—》创建即可。该实验建议选择gpu环境，否则训练十分缓慢。

2. 如果要在自己电脑实验，Paddle Paddle的安装参考一下飞桨的官方文档：

<https://www.paddlepaddle.org.cn/install/quick>，paddlepaddle的版本建议选择2.0.2。

四、实验原理

在神经网络中，线性回归本质上是一个单层的感知机，是一层不带激活函数的全连接网络。全连接神经网络作为神经网络中最简单的网络之一，也可以在前向计算、反向传播后与训练数据完成拟合，公式可表示如下：

$$O = f(Wx + b) \quad (3)$$

其中W，b为模型参数，x为输入数据，O为输出。

五、实验步骤

步骤一 数据加载及预处理

飞桨中提供了读取uci-housing训练集和测试集的接口，分别为

```
1. paddle.text.datasets.UCIHousing(mode='train')
2. paddle.text.datasets.UCIHousing(mode='test')
```

数据集加载后，使用DataLoader类进行封装，DataLoader类的初始化需要指定数据集，并且指定batch_size定制批量迭代的批大小；shuffle参数控制样本顺序，使得不同类型样本可以随机分布；num_workers参数来开启多进程数据加载。该类初始化函数返回一个迭代器，用于训练或测试过程中的批量数据迭代。本实验训练集的batch_size设置为32，测试集的batch_size设置为8。

核心代码如下：

```
1. # 设置paddle 默认的全局数据类型为float64
2. paddle.set_default_dtype('float64')
3. # 加载数据
4. train_dataset = paddle.text.datasets.UCIHousing(mode='train')
5. eval_dataset = paddle.text.datasets.UCIHousing(mode='test')
6. # 封装训练数据
7. train_loader = paddle.io.DataLoader(train_dataset, batch_size=32, shuffle=True)
8. eval_loader = paddle.io.DataLoader(eval_dataset, batch_size=8, shuffle=False)
```

步骤二 模型配置

本实验将会使用 `paddle.nn.Linear(in_features, out_features, weight_attr=None, bias_attr=None, name=None)` 实现线性变化。其中，`in_features`为输入特征的维度，`out_features`为输出特征的维度，`weight_attr`指定权重参数的属性，使用默认的权重参数属性，将权重参数初始化为0，`bias_attr`指定偏置参数的属性，`False`表示不为该层添加偏置，`name`用于网络层输出的前缀标识。

在自定义网络模型时，应当继承 `paddle.nn.Layer` 类，该类基于OOD实现的动态图，实现了训练模型和验证模式，训练模型会执行反向传播，而验证模式不包括反向传播，同时也为dropout等训练、验证时不同的操作提供了支持。在神经网络中，从输入到输出过程称为网络的前向计算，在飞桨中，可用 `forward` 关键字标识，`forward()` 函数中定义网络从前到后的完整计算过程，是实现网络框架最重要的环

节。所以，全连接网络的框架如下：

```
1. class Regressor(paddle.nn.Layer):
2.     def __init__(self):
3.         super(Regressor, self).__init__()
4.         # 定义一层全连接网络，不需要激活函数
5.         .....
6.     def forward(self):
7.         # 前向计算过程
8.         .....
```

步骤三 模型训练。

定义好模型框架后：

1. 首先实例化模型model，在训练阶段，调用model.train()，开启训练模式，该模式具备反向传播功能。

2. 然后定义损失函数，在回归任务中，通常使用均方误差损失函数，paddle.nn.MSELoss(reduction='mean')类提供了计算接口，参数reduction可以是'none'、'mean'、'sum'，设为'none'时不适用约简，设为'mean'时返回loss的均值，设为'sum'时返回loss的和。

3. 定义优化器，神经网络模型的反向传播需要定义传播规则，比如如何计算梯度，每个参数更新多少，更新的速度、幅度等，优化器正是用来实现上述目的的工具。paddle.optimizer中实现了SGD、Adam等多种优化器。使用优化器也需要制定学习率，也就是梯度的更新步长，太大的学习率可能导致网络无法收敛，太小的学习率会使网络收敛很慢。本实验使用paddle.optimizer.SGD(learning_rate=0.001, parameters=None, weight_decay=None, grad_clip=None, name=None)进行优化，其中learning_rate为学习率，parameters指定优化器需要优化的参数，weight_decay为权重衰减系数，grad_clip为梯度裁剪的策略，梯度裁剪可以防止出现梯度爆炸，默认值为None。

4. 定义好模型、损失函数和优化器后，将数据分批次送入模型中后得到输出结果，根据输出结果和label计算loss，然后执行梯度反向传播更新参数(loss.backward())，达到训练目的后，调用paddle.save()保存模型。后续进行预测时，只需要将训练好的模型参数加载到模型中即可。默认迭代200轮(epochs=200)，可以自行调整。

在模型训练过程中不要忘记输出训练损失值，可以每10个批次输出一次，并且

因为在后面模型评估时需要画图记录loss的变化过程，需要定义两个列表：Batch, Batches, all_train_loss = 0, [], [], 在输出loss信息时，也需要将当前批次信息和损失信息加入列表中，核心代码如下：

```
1. Batch = 0
2. # 记录批次
3. Batches = []
4. # 记录损失值，用于后续绘图
5. all_train_loss = []
6.
7. # 模型实例化
8. model = Regressor()
9. # 训练模型
10. model.train()
11. # 均方误差损失函数
12. mse_loss = paddle.nn.MSELoss()
13. # 优化函数 需要传入学习率和模型参数
14. opt = paddle.optimizer.SGD(learning_rate=0.0005, parameters=model.parameters())
15.
16. # 迭代次数
17. epochs_num = 200
18. for pass_num in range(epochs_num):
19.     for batch_id, data in enumerate(train_loader()):
20.         # data 包括 input, label
21.         image = data[0]
22.         label = data[1]
23.         # 前向计算
24.         predict = model(image)
25.         loss = mse_loss(predict, label)
26.         # 10 次迭代记录一次损失值
27.         if batch_id!=0 and batch_id % 10==0:
28.             Batch = Batch + 10
29.             Batches.append(Batch)
30.             # 记录损失值并打印
31.             all_train_loss.append(loss.numpy()[0])
32.             print('epoch:{}, step:{}, train_loss:{}'.format(pass_num, batch_id, loss.numpy()[0]))
33.
34.         # 反向传播
35.         loss.backward()
36.         # 更新参数
37.         opt.step()
38.         # 重置梯度
39.         opt.clear_grad()
40.
41. # 保存模型
42. paddle.save(model.state_dict(), 'Regressor')
```

步骤四 模型评估。

1.模型训练结束后，可以根据保存的损失值的中间结果，绘制损失值随模型迭代次数的变化过程，要用到matplotlib库：



代码如下：

```
1. def draw_train_loss(Batches, train_losses):
2.     title = 'training loss'
3.     plt.title(title, fontsize=24)
4.     plt.xlabel('batch', fontsize=14)
5.     plt.ylabel('loss', fontsize=14)
6.     plt.plot(Batches, train_losses, color='green', label='training loss')
7.     plt.legend()
8.     plt.grid()
9.     plt.show()
```

2. 在测试集上验证模型。首先将先前保存的训练好的参数加载到新的实例化的模型中：

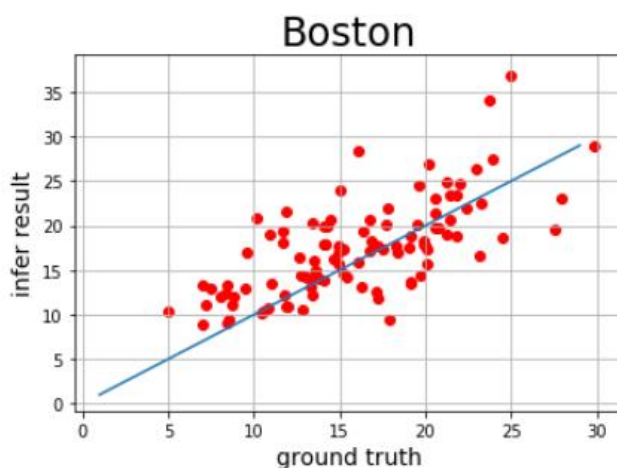
```
1. para_state_dict = paddle.load('Regressor')
2. model = Regressor()
3. model.set_state_dict(para_state_dict)
4. model.eval()
```

然后将测试数据批量输入到网络中进行损失值计算，并计算模型测试集上的损失值，测试代码和训练代码类似，区别是测试中不需要反向传播。

同时需要定义两个列表：`infer_results`, `ground_truth = [], []`，分别为模型输出和真实值的列表。用来绘制模型预测结果与真实值之间的差异。

3.绘制模型预测结果与真实值之间的差异。作为对照，可以观察预测值与真实值构成的坐标点位于 $y=x$ 直线的位置，判断模型性能的好坏，图示如下，点越接近 $y=x$ 直线，说明数据越准确，核心代码如下：

```
1. # 绘制真实值和预测值对比图
2. def draw_infer_result(ground_truths, infer_results):
3.     title = 'Boston'
4.     plt.title(title, fontsize=24)
5.     x = np.arange(1, 30)
6.     y = x
7.     plt.plot(x, y)
8.     plt.xlabel('ground truth', fontsize=14)
9.     plt.ylabel('infer result', fontsize=14)
10.    plt.scatter(ground_truths, infer_results, color='red', label='training cost')
11.    plt.grid()
12.    plt.show()
```



六、实验要求

按照实验步骤编码完成数据的加载、模型的配置、模型的训练和评估，并绘制合适的图像描述模型的训练过程和模型的评估结果，绘制的图像合理即可，无需一致。

实验四 基于Llama 3实现商品评论情感分析

一、实验目的

本实验旨在通过基于Llama 3模型实现商品评论情感分析，深入理解和掌握处理大模型的技术。学习如何获取和预处理数据集，配置适合的软硬件环境，加载和微调预训练的大型语言模型，以及评估和分析模型的性能。

二、实验内容

本实验使用来自modelscope的[商品评论情感预测数据集](#)，该数据集包含从2011年1月1日到2014年3月31日（3年多）某电商网站的消费者购买行为，用户评分，评论和产品元数据，涵盖15个一级产品类别，987个二级产品类别，近2个百万用户，超过10万种产品和超过6,000万条评论。该数据集中的每个文本评论都包含三个子评论：正面评论，负面评论和整体评论。其中训练数据集45366条，测试数据集5032条。

本实验要求使用LlamaFactory搭建Llama 3模型，并利用上述数据集训练该模型；在训练过程中，采用LoRA微调方法对模型进行优化；训练结束后，保存模型并存储模型的参数；评估网络时，从参数文件中加载保存的模型参数，再进行模型评估。通过这种方式，能够利用Llama3模型的强大性能，并通过LoRA微调提升模型在特定任务上的效果。

三、实验环境

1.硬件环境：

- CPU：Intel i7-12700K
- GPU：NVIDIA RTX 4090
- 内存：64GB
- 存储：1TB SSD

2.软件环境：

- 操作系统：Ubuntu 20.04
- Python版本：3.8
- 主要库：PyTorch、Transformers、Datasets
- Llama3模型：通过huggingface平台获取并使用LlamaFactory部署

建议使用autoDL平台进行实验，该平台提供了良好的算力支持，并且已经集成

了所需的深度学习框架（如PyTorch等）。可以通过以下步骤配置环境：

1. 登录autoDL平台并创建一个项目。
2. 选择适合的GPU环境，以确保训练过程的高效性，避免训练速度过慢。
3. 创建一个Python项目，安装并配置所需的库（如PyTorch、Transformers）。
4. 使用autoDL平台提供的资源进行实验，确保软硬件环境符合模型训练要求。

四、实验原理

Llama 3 是一种基于 Transformer 架构的预训练语言模型，旨在通过大规模文本数据的预训练学习语言的深层次语义和结构特征。Transformer 架构自提出来，因其独特的自注意力机制（Self-Attention）而在自然语言处理任务中取得了优异的成绩。Llama3 作为 Transformer 架构的一种变种，依托其强大的文本理解和生成能力，能够在情感分析、文本生成、问答等多种自然语言处理任务中展现出出色的表现。

Transformer 架构的核心在于自注意力机制，允许模型在处理每个输入单词时，关注到句子中其他位置的单词，从而捕捉到全局的上下文信息。这种机制是通过以下公式实现的：

1. Transformer 编码器由多个层叠的自注意力机制和前馈神经网络组成。每一层的输入是一个包含序列中每个单词信息的向量，经过自注意力机制，模型会为每个单词分配一个权重，用以决定该单词在理解其他单词时的重要性。自注意力机制的计算公式为：

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

其中，Q、K、V 分别是查询（Query）、键（Key）和值（Value）向量， d_k 是键的维度，softmax 函数用于对权重进行归一化处理。通过自注意力机制，模型能够在编码每个单词时，考虑到其他所有单词的信息。

2. 预训练与微调：Llama3 模型通过大规模文本数据进行预训练，使用目标任务相关的损失函数（例如交叉熵损失），从而学习到语言的通用特征。在进行情感分析等任务时，模型会根据预训练过程中学到的语言表示进行微调，通过标注数据进一步优化模型，使其能够准确地分类不同的情感。

3. LoRA 微调：LoRA（Low-Rank Adaptation）是一种用于预训练模型微调的技术，它通过对模型参数的低秩分解来减少微调过程中的计算开销。具体来说，LoRA 在保持大部分预训练模型参数不变的情况下，引入低秩矩阵来调整模型的某

些部分，从而使模型能够更好地适应目标任务。LoRA 通过以下公式实现：

$$\Delta W = A \cdot B$$

其中，A 和 B 是低秩矩阵， ΔW 表示调整后的权重矩阵。低秩矩阵的引入使得模型能够在保持预训练知识的同时，针对特定任务进行高效的微调。

五、实验步骤

步骤一 数据加载。

在数据加载部分，我们使用了 ModelScope 提供的 MsDataset 类来加载数据集。在代码中，首先通过 MsDataset.load 方法加载了 'DAMO_NLP/jd' 数据集的训练集，并将其存储在 ds_train 变量中。这里指定了数据集的子集名称为 'default'，并选择了 split='train' 来获取训练数据。同样，使用 MsDataset.load 方法加载了数据集的验证集，存储在 ds_val 变量中，以供后续的模式评估。通过这种方式，我们可以方便地获取不同数据集分割（训练集和验证集）并进行相应的处理。（如果数据集下载失败也可在 modelscope 官网直接下载）

```
1. from modelscope.msdatasets import MsDataset
2. ds_train = MsDataset.load('DAMO_NLP/jd', subset_name='default', split='train')
3.
4. from modelscope.msdatasets import MsDataset
5. ds_val = MsDataset.load('DAMO_NLP/jd', subset_name='default', split='validation')
6.
```

步骤二 数据处理。

下载后的数据集无法直接应用到微调，我们还需要结合提示工程，将数据集转化为大模型微调所需要的格式（即问答对的形式）

数据转化代码如下：

```
1. import json
2. from modelscope.msdatasets import MsDataset
3. from tqdm import *
4. ds_train = MsDataset.load('DAMO_NLP/jd', subset_name='default', split='train')
5. ds_val = MsDataset.load('DAMO_NLP/jd', subset_name='default', split='validation')
6. print(len(ds_train["sentence"]))
7. print(len(ds_val["sentence"]))
8. outout = []
```

```

9. SYSTEM_PROMPT = "我在做商品评论情感预测，需根据用户评价判断是好评还是差评，其中输出 0 代表差评，输出 1 代表好评，请严格保证输出结果为整数并且只能是 0 或者 1。输入的用户评价为： "
10. for i in tqdm(range(len(ds_val["sentence"]))):
11.     sentence = ds_val["sentence"][i]
12.     if (ds_val["label"][i] == None or ds_val["sentence"][i] == None):
13.         continue
14.     label = str(int(ds_val["label"][i]))
15.     outout.append({"instruction":SYSTEM_PROMPT+sentence,"input":"","output":label})
16. with open("jd_val.json", "w") as json_file:
17.     json.dump(outout, json_file,ensure_ascii=False)

```

步骤三 模型配置。

1. 下载基于中文数据训练过的 Llama 3 8B 模型：[shenzhi-wang/Llama3-8B-Chinese-Chat](#)。

（可选）配置 hf 国内镜像站：

```

1. export HF_ENDPOINT=https://hf-mirror.com
2.
3. pip install huggingface-cli
4.
5. huggingface-cli download --resume-download shenzhi-wang/Llama3-8B-Chinese-Chat --local-dir /root/autodl-tmp/models/Llama3-8B-Chinese-Chat1

```

第五行命令解释：该命令从 Hugging Face Hub 下载 shenzhi-wang/Llama3-8B-Chinese-Chat 模型，并将其保存到本地目录 /root/autodl-tmp/models/Llama3-8B-Chinese-Chat1。如果下载过程被中断，使用 --resume-download 选项会让下载从中断点继续。

2. 安装 LLaMA-Factory 框架

```

1. git clone https://github.com/hiyouga/LLaMA-Factory.git
2. cd LLaMA-Factory
3. pip install -e .

```

4. 准备训练数据和验证数据：

将处理好的训练数据和验证数据分别放在 LLaMA-Factory/data/ jd_train.json，LLaMA-Factory/data/ jd_val.json
并且修改数据注册文件：LLaMA-Factory/data/dataset_info.json（即将下面代码放入 dataset_info.json 中）

```

1.  "train": {
2.    "file_name": "jd_train.json",
3.    "columns": {
4.      "prompt": "instruction",
5.      "query": "input",
6.      "response": "output"
7.    }
8.  },
9.  "validation": {
10.    "file_name": "jd_val.json",
11.    "columns": {
12.      "prompt": "instruction",
13.      "query": "input",
14.      "response": "output"
15.    }

```

5.启动Web UI

```

1. cd LLaMA-Factory
2. llamafactory-cli webui

```

步骤四 模型训练

1.训练可以采用Web UI页面训练。

```

1. 1. cd LLaMA-Factory
2. 2. llamafactory-cli webui

```

2.也可以采用命令行的方式训练，在LLaMA-Factory目录下构建cust/train_llama3_lora_sft.yaml，train_llama3_lora_sft.yaml内容如下所示：

```

1.  CUDA_VISIBLE_DEVICES=0 llamafactory-cli train \
2.    --stage sft \
3.    --do_train True \
4.    --model_name_or_path /home/guo/hub/Qwen1___5-7B-Chat \ #选择大
    模型下载位置
5.    --preprocessing_num_workers 16 \
6.    --finetuning_type lora \
7.    --template qwen \
8.    --flash_attn auto \
9.    --dataset_dir data \
10.   --dataset jd \ #设置为你的数据集
11.   --cutoff_len 1024 \
12.   --learning_rate 5e-05 \
13.   --num_train_epochs 3.0 \
14.   --max_samples 100000 \
15.   --per_device_train_batch_size 2 \
16.   --gradient_accumulation_steps 8 \
17.   --lr_scheduler_type cosine \
18.   --max_grad_norm 1.0 \

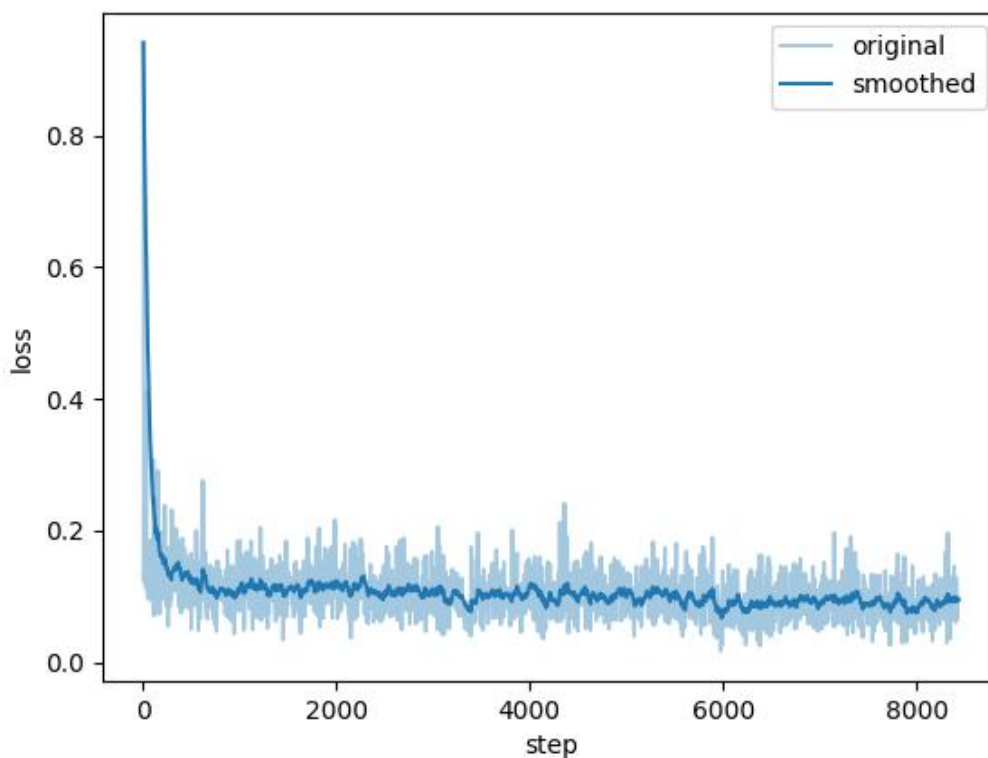
```

```
19. --logging_steps 5 \  
20. --save_steps 100 \  
21. --warmup_steps 0 \  
22. --optim adamw_torch \  
23. --packing False \  
24. --report_to none \  
25. --output_dir saves/Qwen1.5-7B-Chat/lora/train_2024-05-23-14-32-35 \  
26. --fp16 True \  
27. --plot_loss True \  
28. --lora_rank 8 \  
29. --lora_alpha 16 \  
30. --lora_dropout 0 \  
31. --lora_target q_proj,v_proj
```

构建好后在命令行中执行下面的命令就会开始训练模型：

```
1. llamafactory-cli train cust/train_llama3_lora_sft.yaml
```

模型训练完后得到loss图像如下：



步骤五 模型评估与预测。

LLaMA-Factory也支持用web界面的方式评估和预测，具体评测使用方式如下所示。



评测结束后，得到一个generated_predictions.jsonl，部分内容如下：

```
1. {"label": "1", "predict": "1"}
2. {"label": "0", "predict": "0"}
3. {"label": "1", "predict": "1"}
4. {"label": "1", "predict": "1"}
5. {"label": "0", "predict": "0"}
6. {"label": "1", "predict": "1"}
7. {"label": "1", "predict": "1"}
8. {"label": "0", "predict": "0"}
9. {"label": "0", "predict": "0"}
10. {"label": "0", "predict": "0"}
11. {"label": "0", "predict": "0"}
12. {"label": "0", "predict": "0"}
13. {"label": "0", "predict": "0"}
14. {"label": "0", "predict": "0"}
15. {"label": "0", "predict": "0"}
16. {"label": "1", "predict": "1"}
17. ...
```

编写准确率计算代码 $Acc=(TP+TN)/(TP+TN+FP+FN)$ ，代码如下：

```
1. import json
2.
3. def calculate_accuracy(json_file):
4.     correct = 0
5.     total = 0
6.
7.     with open(json_file, 'r', encoding='utf-8') as f:
8.         for line in f:
9.             data = json.loads(line)
10.            label = data["label"]
11.            predict = data["predict"]
12.
13.            # 如果标签和预测值相等，表示预测正确
```

```
14.         if label == predict:
15.             correct += 1
16.             total += 1
17.
18.     # 计算准确率
19.     accuracy = correct / total if total > 0 else 0
20.     return accuracy
21.
22. # 使用示例
23. json_file = '/root/project/output/eval_2024-10-15-08-19-
24.             50/generated_predictions.jsonl'
25. accuracy = calculate_accuracy(json_file)
26. print(f'微调后模型的准确率为: {accuracy * 100:.2f}%')
```

六、实验要求

1.按照实验步骤编码完成数据的加载和预处理、模型的配置、模型的训练和评估，并得到模型的训练过程的损失图像和模型训练前后的评估结果（以准确率为指标）。

2.最终在测试集中抽取一条数据输入到模型中，最后输出该数据的文本以及该文本的情感（好评或者差评）。

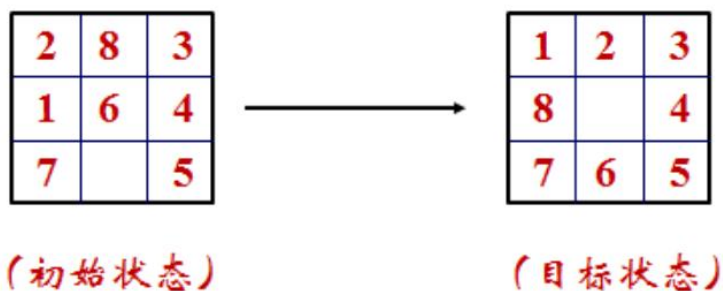
实验五 A*算法之八数码问题

一、实验目的

熟悉和掌握启发式搜索的定义、估价函数和搜索算法，并利用A*算法求解八数码问题，加深对图搜索算法的理解，尤其是对启发式搜索基本原理的理解，并能够解决一些应用问题。

二、实验内容

在 3×3 的棋盘上，摆有八个棋子，每个棋子上标有1至8的某一数字，称为数码。棋盘中留有一个空格，空格用0表示。空格周围的数码可以移到空格中。要求解的问题是：给出一种初始布局（初始状态）和目标布局（为了使题目简单，设目标状态为123、804、765），如图1所示，找到一种数码移动次数最少的方法，实现从初始布局到目标布局的转变。通过设计启发函数，编程实现求解。如果问题有解，给出数码移动过程，否则，报告问题无解（可以通过限定时间阈值或步骤阈值）。



三、实验环境

推荐使用Python语言，编译器推荐使用PyCharm 2021.2.1(Professional Edition)，环境如下：

```
python=3.8.0
prettytable=3.1.1
```

四、实验原理

A*算法是一种启发式图搜索算法，其特点在于对估价函数的定义上。对于一般的启发式图搜索，总是选择估价函数 f 值最小的节点作为扩展节点。因此， f 是根据需要找到一条最小代价路径的观点来估算节点的，所以，在八数码问题中，可考虑每个节点 n 的估价函数值为两个分量：从起始节点到节点 n 的实际代价 $d(n)$ (搜索树的深度，也可以理解为当前是第几轮循环)以及从节点 n 到达目标节点的估价代价 $w(n)$ (即结点 n 与目标结点相比较放错位置的数码个数)。节点 n 的一种估价函数 $f(n)$ 如

下公式所示。

$$f(n) = d(n) + w(n) \quad (1)$$

五、实验步骤

1、状态空间的定义

本题的状态空间是一个3*3的九宫格,里面标记有1-8个数字的数码和一个空格。为了方便表示,空格用0来表示,状态空间用下列数组表示(这里使用numpy来表示)

```
1. import numpy as np
2. start_data = np.array([[2, 8, 3], [1, 6, 4], [7, 0, 5]]) # 初始状态
3. end_data = np.array([[1, 2, 3], [8, 0, 4], [7, 6, 5]]) # 目标状态
```

2、各种操作的定义

对于操作,可以理解为更改状态空间的一些规则,从空格0的移动来考虑,操作包括空格上移、空格下移、空格左移和空格右移。同时,在进行移动空格之前还需要通过一些约束条件进行判断,例如图所示的状态中,空格0是无法向下移动的。

2	8	3
1	6	4
7		5

例如左移操作:

```
1. x, y = find_zero(num_data) # 空格(0)的位置
2. num = np.copy(num_data) # 当前状态
3. if direction == 'left':
4.     if y == 0:
5.         # print('不能左移')
6.         return num
7.     num[x][y] = num[x][y - 1]
8.     num[x][y - 1] = 0
9.     return num
10. # 找空格(0)号元素在哪的函数
11. def find_zero(num):
12.     tmp_x, tmp_y = np.where(num == 0)
13.     # 返回0所在的x坐标与y坐标
14.     return tmp_x[0], tmp_y[0]
```

3、A*算法框架定义

```
1. import queue
2. #步骤1. 定义Node类
3. #步骤2. 定义opened表和closed表, 创建初始节点, 并且加入opened表
```

```

中
4. def method_a_function():
5.     while len(opened.queue) != 0:
6.         # 步骤3. 取队首元素节点
7.         # 步骤4. 判断当前节点的 data 是否为目标值, 是则返回正确值
8.         # 步骤5. 将取出的节点加入 closed 表中
9.         # 步骤6. 对当前节点 data 中的空格 0 执行左、右、上、下移动四个操作, 创建当前节点的一切后继子节点, 并对 opened 表进行更新
10.        for action in ['left', 'right', 'up', 'down']:
11.            # 创建子节点
12.            # 判断是否在 closed 表中, 如果不在 close 表中, 将其加入
opened 表(并且考虑到与 opened 表中已有元素重复的更新情况)
13.            # 步骤7. 根据节点的 f_loss 值, 为 opened 表进行排序 (f_loss
值从小到大)
14.        # 步骤8. 将初始状态到目标状态所经过的所有节点进行输出

```

步骤1核心代码:

```

1. # Node 类 (包含当前数据内容, 父节点, 步数)
2. class Node:
3.     f_loss = -1 # 启发值
4.     step = 0 # 初始状态到当前状态的距离 (步数)
5.     parent = None, # 父节点
6.     # 用状态和步数构造节点对象
7.     def __init__(self, data, step, parent):
8.         self.data = data # 当前状态数值
9.         self.step = step
10.        self.parent = parent
11.        # 计算 f(n) 的值
12.        self.f_loss = cal_wcost(data) + step
13. # 计算 w(n) 的函数
14. def cal_wcost(num):
15.     '''
16.     计算 w(n) 的值, 及放错元素的个数
17.     :param num: 要比较的数组的值
18.     :return: 返回 w(n) 的值
19.     '''
20.     con = 0
21.     for i in range(3):
22.         for j in range(3):
23.             tmp_num = num[i][j]
24.             compare_num = end_data[i][j]
25.             if tmp_num != 0:
26.                 con += tmp_num != compare_num
27.     return con

```

步骤2核心代码:

```

1. opened = queue.Queue() # opened 表
2. start_node = Node(start_data, 0, None)
3. opened.put(start_node)
4. closed = {} # close 表

```

步骤3核心代码:

```
1. node = opened.get()
```

步骤4核心代码:

```
1. if (node.data == end_data).all():
2.     print(f'总共耗费{con}轮')
3.     return node
```

步骤5核心代码:

```
1. closed[data_to_int(node.data)] = 1
2. # 节点 data 转换函数, closed 表为一个字典, 因为它的键不能放
   numpy.array,
3. # 所以将 numpy 的数组转换为一个 int 类型的数子, 这里的函数类似于 hash
4. # 函数
5. def data_to_int(num):
6.     value = 0
7.     for i in num:
8.         for j in i:
9.             value = value * 10 + j
10.    return value
```

步骤6核心代码:

```
1. for action in ['left', 'right', 'up', 'down']:
2.     child_node = Node(swap(node.data, action), node.step + 1, node)
3.     index = data_to_int(child_node.data)
4.     if index not in closed:
5.         refresh_opened(child_node)
6. # 交换位置的函数 移动的时候要判断一下是否可以移动(是否在底部)
7. # 记空格为 0 号, 则每次移动一个数子可以看成对空格(0)的移动, 总共有四种可能
8. def swap(num_data, direction):
9.     x, y = find_zero(num_data)
10.    num = np.copy(num_data)
11.    if direction == 'left':
12.        if y == 0:
13.            # print('不能左移')
14.            return num
15.        num[x][y] = num[x][y - 1]
16.        num[x][y - 1] = 0
17.        return num
18.    if direction == 'right':
19.        if y == 2:
20.            # print('不能右移')
21.            return num
```

```

22.         num[x][y] = num[x][y + 1]
23.         num[x][y + 1] = 0
24.         return num
25.     if direction == 'up':
26.         if x == 0:
27.             # print('不能上移')
28.             return num
29.         num[x][y] = num[x - 1][y]
30.         num[x - 1][y] = 0
31.         return num
32.     if direction == 'down':
33.         if x == 2:
34.             # print('不能下移')
35.             return num
36.         else:
37.             num[x][y] = num[x + 1][y]
38.             num[x + 1][y] = 0
39.             return num
40. # opened 表更新函数, 比较当前节点是否在 opened 表中, 如果在, 根据 f(n)
    的大小来判断去留
41. def refresh_opened(now_node):
42.     '''
43.     :param now_node: 当前的节点
44.     :return:
45.     '''
46.     tmp_opened = opened.queue.copy() # 复制一份 opened 表的内容
47.     for i in range(len(tmp_opened)):
48.         '''这里要比较一下 node 和 now_node 的区别, 并决定是否更新'''
49.         data = tmp_opened[i]
50.         now_data = now_node.data
51.         if (data == now_data).all():
52.             data_f_loss = tmp_opened[i].f_loss
53.             now_data_f_loss = now_node.f_loss
54.             if data_f_loss <= now_data_f_loss:
55.                 return False
56.             else:
57.                 print('')
58.                 tmp_opened[i] = now_node
59.                 opened.queue = tmp_opened # 更新之后的 opened 表
    还原
60.         return True
61.     tmp_opened.append(now_node)
62.     opened.queue = tmp_opened # 更新之后的 opened 表还原
63.     return True

```

步骤7核心代码:

```

1.     sorted_by_floss()
2.     # opened 表排序函数
3.     def sorted_by_floss():

```

```

4.     tmp_opened = opened.queue.copy()
5.     length = len(tmp_opened)
6.     # 排序, 从小到大, 当一样的时候按照 step 的大小排序
7.     for i in range(length):
8.         for j in range(length):
9.             if tmp_opened[i].f_loss < tmp_opened[j].f_loss:
10.                tmp = tmp_opened[i]
11.                tmp_opened[i] = tmp_opened[j]
12.                tmp_opened[j] = tmp
13.            if tmp_opened[i].f_loss == tmp_opened[j].f_loss:
14.                if tmp_opened[i].step > tmp_opened[j].step:
15.                    tmp = tmp_opened[i]
16.                    tmp_opened[i] = tmp_opened[j]
17.                    tmp_opened[j] = tmp
18.     opened.queue = tmp_opened

```

步骤8核心代码:

```

1.  result_node = method_a_function()
2.  import prettytable as pt
3.  # 获取路径中所有节点的函数, 依次获取目标节点的父节点, 形成一条正确
    顺序的路径, 然后使用循环将这条路径输出
4.  def output_result(node):
5.      all_node = [node]
6.      for i in range(node.step):
7.          father_node = node.parent
8.          all_node.append(father_node)
9.          node = father_node
10.     return reversed(all_node)
11. node_list = list(output_result(result_node))
12. tb = pt.PrettyTable() # 用来生成美观的ASCLL 格式的表格
13. tb.field_names = ['step', 'data', 'f_loss']
14. for node in node_list:
15.     num = node.data
16.     tb.add_row([node.step, num, node.f_loss])
17.     if node != node_list[-1]:
18.         tb.add_row(['---', '-----', '---'])
19. print(tb)

```

六、实验要求

1 参考A*算法核心代码, 以八数码问题为例实现A*算法的求解程序(编程语言不限), 要求设计两种不同的估价函数(例如以搜索树的深度作为实际代价, 用当前状态到目标状态的manhattan距离作为启发函数)。

提示1: 启发函数设计可参考如下方案:

- (1) $h(n)=w(n)$, $w(n)$ 代表状态 n 与目标状态相比, 位置不符的数码数目。
- (2) $h(n)=P(n)$, $p(n)$ 是状态 n 中每个数码离家(在 sg 中的位置)的最短距离。

(3) $h(n)=p(n)+3s(n)$, $s(n)$ 计算方法: 沿周围非中心方格依顺时针方向检查状态 n 的每一个数码, 如果其后紧跟的数码是目标状态中该数码的后继者, 该数码得0分, 否则得2分; 若中心方格有数码得1分, 否则得0分。

提示2: 估价函数可设计为: $f(n)=d(n)+h(n)$, 其中, $d(n)$ 表示节点深度, $h(n)$ 可取设计方案(1)、(2)或(3)。

2 在求解八数码问题的A*算法中, 设置相同的初始状态和目标状态, 针对不同的估价函数, 求得问题的解, 并比较它们对搜索算法性能的影响, 包括扩展节点数、生成节点数等。

3 对于八数码问题, 设置与上述2相同的初始状态和目标状态, 用广度优先搜索算法/深度优先搜索算法求得问题的解, 比较搜索过程中的扩展节点数、生成节点数。

4 提交源程序(电子版)和实验报告(纸质版)。

实验六 归结原理实验

一、实验目的

加深学生对归结原理进行定理证明过程的理解，掌握基于谓词逻辑的归结过程中子句变换过程、替换与合一算法和归结策略等重要环节，进一步了解实现机器自动定理证明的步骤。

二、实验内容

1. 输入一个符号串，判断其是否为谓词公式（可选）。
2. 对于任意一阶谓词逻辑描述的定理，给出基于归结原理的证明过程。如输入：

$$A1 : (\forall x) (P(x) \rightarrow (Q(x) \wedge R(x)))$$

$$A2 : (\forall x) (P(x) \wedge S(x))$$

$$G : (\exists x) (S(x) \wedge R(x))$$

证明：G是A1和A2的逻辑结果。

三、实验环境

推荐使用C++语言，编译器推荐使用Microsoft Visual Studio Professional2019 (version=16.3.8)。

四、实验原理

归结原理是一种推理规则。从谓词公式转化为子句集的过程中看出，在子句集中子句之间是合取关系，其中只要有一个子句不可满足，则子句集就不可满足。若一个子句集中包含空子句，则这个子句集一定是不可满足的。归结原理就是基于这一认识提出来的。

应用归结原理进行定理证明的过程：

设要被证明的定义表示为：

$$A_1 \wedge A_2 \wedge \cdots \wedge A_n \rightarrow B$$

(1) 首先否定结论B，并将否定后的公式 $\neg B$ 与前提公式集组成如下形式的谓词公式： $G = A_1 \wedge A_2 \wedge \cdots \wedge A_n \wedge \neg B$ 。

(2) 求谓词公式G的子句集S。

(3) 应用归结原理，证明子句集S的不可满足性。

五、实验步骤

步骤一 设计谓词公式的存储结构，即内部表示，注意对全称量词 $\forall x$ 和存在量词 $\exists x$ 可采用其他符号代替。

原符号	替换符号
全称量词 $\forall$	@
存在量词 $\exists$	#
析取符号 $\vee$	
合取符号 $\wedge$	&
取反符号 $\neg$	~
蕴含符号 $\rightarrow$	>

例如A1： $(\forall x)(P(x) \rightarrow (Q(x) \wedge R(x)))$ ，输入的时候要替换为A1：
 $(@x)(P(x) > (Q(x) \& R(x)))$ 。

核心代码如下所示：

```

1. namespace FormulaNamespace {
2.     // 公式符号定义
3.     const char EQ = '#'; // 存在量词符号
4.     const char UQ = '@'; // 全称量词符号
5.     const char IMPLICATION = '>'; // 蕴含符号
6.     const char NEGATION = '~'; // 否定符号
7.     const char CONJUNCTION = '&'; // 合取符号
8.     const char DISJUNCTION = '|'; // 析取符号
9.     const char CONSTANT_ALPHA[] = { 'a', 'b', 'c', 'd', 'e', 'i', 'j', 'k' };
10. }
11. #endif // SUBSENTENCE_H

```

步骤二 变换子句集，可按以下过程变换，变换过程的核心代码如下所示：

1、消去蕴含连接词。

```

1. Formula& RemoveImplication(Formula& f)
2. {
3.     FormulaIter iter;
4.     while((iter = find(f.begin(), f.end(), IMPLICATION))
5.           != f.end()) {
6.         *iter = DISJUNCTION; // 将蕴含符号替换为析取符号
7.         FormulaRevIter revIter(iter);
8.         revIter =
9.             GetBeginOfFormula(revIter, f.rend()); // 查找蕴含前件
10.        iter = revIter.base()-1; // 反向迭代器到正向迭代器转换需
        要减 1
11.        f.insert(iter, NEGATION); // 在前件前面插入否定
12.    }
13.    return f;

```

14. }

2、将否定符号移到紧靠谓词的位置。

```

1. Formula& MoveNegation(Formula& f)
2. {
3.     FormulaIter iter = find(f.begin(), f.end(), NEGATION);
4.     while(iter != f.end()) {
5.         if(*(iter+1) == '(') { // 否定不是直接修饰谓词公式，需要内移
6.             // 否定符号在量词前
7.             if(*(iter+2) == EQ || *(iter+2) == UQ) {
8.                 // 量词取反
9.                 *(iter+2) == EQ ? *(iter+2) = UQ : *(iter+2) = EQ;
10.
11.             string leftDonePart(f.begin(), iter+5);
12.             // 移除否定符号
13.             leftDonePart.erase(find(leftDonePart.begin(),
14.                                     leftDonePart.end(), NEGATION));
15.
16.             string rightPart(iter + 5, f.end());
17.             // 否定内移
18.             rightPart.insert(rightPart.begin(), NEGATION);
19.             // 递归处理右部分
20.             MoveNegation(rightPart);
21.             string(leftDonePart + rightPart).swap(f);
22.             return f;
23.         } else { // 修饰着多个公式，形如~(P(x)|Q(x))
24.             iter = f.insert(iter+2, NEGATION); // 内移否定符号
25.             while(1) {
26.                 iter = FindFormula(iter, f.end());
27.                 assert(iter != f.end() && "No Predicate Formula!");
28.                 FormulaIter iter2 = FindPairChar(iter, f.end(), '(', ')');
29.                 ++iter2;
30.                 if(IsConnector(*iter2)) {
31.                     *iter2==DISJUNCTION ? *iter2=CONJUNCTION: *iter2 = DISJUNC
TION;
32.                     iter = f.insert(iter2+1, NEGATION);
33.                 } else
34.                     break;
35.             }
36.             f.erase(find(f.begin(), f.end(), NEGATION)); // 清除原否定符号
37.             return MoveNegation(f);
38.         }
39.     } else if(*(iter+1) == NEGATION) { // 两个否定，直接去掉
40.         f.erase(iter, iter + 2);
41.         return MoveNegation(f); // 重新处理
42.     } else {
43.         iter = find(iter + 1, f.end(), NEGATION);
44.     }
45. }
46. return f;

```

```
47. }
```

3、适当改名使量词间不合同名指导变元，对变元标准化。

```
1. Formula& StandardizeValues(Formula& f)
2. {
3.     set<char> checkedAlpha;
4.     FormulaIter iter = FindQuantifier(f.begin(), f.end());
5.     while(iter != f.end()) {
6.         char varName = *++iter; // 获取变量名
7.         if(checkAlpha.find(varName) == checkedAlpha.end()) {
8.             checkedAlpha.insert(varName);
9.         }else { // 变量名冲突，需要改名
10.            // 获取新名称
11.            char newName = FindNewLowerAlpha(checkedAlpha);
12.            // 查找替换右边界
13.            FormulaIter rightBorder = FindPairChar(iter + 2, f.end(), '('
14.            ', ')');
15.            // 将冲突变量名替换为新的名称
16.            *iter = newName;
17.            replace(iter, rightBorder, varName, newName);
18.            iter = rightBorder; // 移动到新的开始
19.        }
20.        iter = FindQuantifier(iter, f.end());
21.    }
22.    return f;
23. }
```

4、化为前束范式。

```
1. Formula& TransformToPNF(Formula& f)
2. {
3.     FormulaIter iter = FindQuantifier(f.begin(), f.end());
4.     if(iter == f.end())
5.         return f;
6.     else if(iter-1 == f.begin()) { // 量词已经在最前面
7.         iter += 3;
8.         string leftPart(f.begin(), iter);
9.         string rightPart(iter, f.end());
10.        TransformToPNF(rightPart); // 递归处理右部分
11.        (leftPart + rightPart).swap(f);
12.    }else { // 量词在内部，需要提到前面
13.        string quantf(iter-1, iter+3); // 保存量词
14.        f.erase(iter-1, iter+3); // 移除量词
15.        f.insert(f.begin(), quantf.begin(), quantf.end());
16.        return TransformToPNF(f); // 继续处理
17.    }
18.    return f;
19. }
```

5、化为Skolem标准型（将母式化为合取范式）。

6、消去存在量词。

```

1. Formula& RemoveEQ(Formula& f)
2. {
3.     set<char> checkedAlpha;
4.     FormulaIter eqIter = find(f.begin(), f.end(), EQ);
5.     if(eqIter == f.end()) return f;
6.     FormulaRevIter uqIter = find(FormulaRevIter(eqIter), f.rend(),
        UQ);
7.     if(uqIter == f.rend()) { // 该存在量词前没有全称量词
8.         char varName = *(eqIter + 1);
9.         char newName = GetNewConstantAlfa(f);
10.        auto rightBound = FindPairChar(eqIter + 3, f.end(), '(', ')');
11.        ;
12.        assert(rightBound != f.end());
13.        replace(eqIter + 3, rightBound, varName, newName); // 常量化
14.        f.erase(eqIter - 1, eqIter + 3); // 移除存在量词
15.    }else {
16.        // 记录公式中已经存在的变元名称
17.        copy_if(f.begin(), f.end(),
18.            inserter(checkedException, checkedAlpha.begin()),
19.            ptr_fun<int, int>(isalpha));
20.        const char oldName = *(eqIter+1);
21.        // 准备任意量词的函数来替换该存在量词
22.        const char funcName = FindNewLowerAlpha(checkedException);
23.        string funcFormula;
24.        funcFormula = funcFormula + funcName + '(' + *(uqIter-
25.            1) + ')';
26.        f.erase(eqIter - 1, eqIter + 3); // 移除存在量词
27.        ReplaceAlphaWithString(f, oldName, funcFormula);
28.    }
29.    RemoveOuterBracket(f);
30.    return RemoveEQ(f); // 递归处理
31. }

```

7、消去全称量词。

```

1. Formula& RemoveUQ(Formula& f)
2. {
3.     FormulaIter uqIter = find(f.begin(), f.end(), UQ);
4.     while(uqIter != f.end()) {
5.         uqIter = f.erase(uqIter-1, uqIter+3); // 直接移除全称量词
6.         uqIter = find(uqIter, f.end(), UQ); // 继续扫描
7.     }
8.     RemoveOuterBracket(f);
9.     return f;

```

```
10. }
```

8、消去合取词，以子句为元素组成一个集合S。

```
1. void ExtractSubsentence(SubsentenceSet& subset,const Formula& f
   )
2. {
3.     auto leftIter = f.begin(),middleIter = find(f.begin(), f.end()
   , CONJUNCTION);
4.     while(middleIter != f.end()) {
5.         subset.insert(string(leftIter, middleIter));
6.         leftIter = middleIter + 1;
7.         middleIter = find(middleIter + 1, f.end(), CONJUNCTION);
8.     }
9.     subset.insert(string(leftIter, middleIter));
10. }
```

需要考虑子句、子句集的存储结构的设计。

步骤三 选择并设计归结策略，常用的归结策略有：

删除策略、支持集策略、线性归结策略、输入归结策略、单元归结策略、锁归结策略、祖先过滤型策略等。

步骤四 实现归结算法，并在其中实现合一算法，使用归结原理进行定理证明，要求采用归结反演过程，即：

1. 先求出要证明的命题公式的否定式的子句集S；
2. 然后对子句集S（一次或者多次）使用归结原理；
3. 若在某一步推出了空子句，即推出了矛盾，则说明子句集S是不可满足的，从而原否定式也是不可满足的，进而说明原公式是永真的。

合一算法及核心代码如下：

1. 置 $k=0$, $S_k=S$, $\sigma_k = \varepsilon$;
2. 若 S_k 只含有一个谓词公式，则算法停止， σ_k 就是最一般合一；
3. 求 S_k 的差异集 D_k ；
4. 若中存在元素 x_k 和 t_k ，其中 x_k 是变元， t_k 是项且 x_k 不在 t_k 中出现，则置 $S_{k+1}=S_k\{t_k/x_k\}$, $\sigma_k = \varepsilon$ ，转Step2；
5. 算法停止，S的最一般合一不存在。

```
1. S substitutes MGU(const FormulaNamespace::Subsentence& subsent1,
   const FormulaNamespace::Subsentence& subsent2)
2. {
3.     {
4.         pair<Subsentence, Subsentence> w = { subsent1, subsent2 };
```

```

5.    S substitutes mgu;
6.    while(w.first != w.second) { // w 未合一
7.        // 找不一致集
8.        auto iter1 = FindPredicate(w.first.begin(), w.first.end());
9.        auto iter2 = FindPredicate(w.second.begin(), w.second.end());
10.       while(iter1 != w.first.end() && iter2 != w.second.end()) {
11.           if(*iter1 != *iter2)
12.               break;
13.           iter1 = FindPredicate(iter1 + 1, w.first.end());
14.           iter2 = FindPredicate(iter2 + 1, w.second.end());
15.       }
16.       // 找到不一致集合
17.       if(iter1 != w.first.end() && iter2 != w.second.end()) {
18.           string item1 = GetPredicate(iter1, w.first.end());
19.           string item2 = GetPredicate(iter2, w.second.end());
20.           // 不允许置换有嵌套关系
21.           if(StrStr(item1, item2) != item1.end() ||
22.              StrStr(item2, item1) != item2.end()) {
23.               throw ResolutionException("cannot unifier");
24.           }
25.           // 只允许常量替换变量
26.           if(!IsConstantAlpha(*iter1))
27.               item1.swap(item2);
28.           // 更新置换, 然后置换子句集
29.           S substitutes sustiSet = { make_pair(item1, item2) };
30.           mgu = ComposeSustitutes(mgu, sustiSet);
31.           Substitution(w.first, mgu);
32.           Substitution(w.second, mgu);
33.       }
34.       else { // 两子句不可合一
35.           throw ResolutionException("cannot unifier");
36.       }
37.     }
38.     return mgu;
39. }

```

步骤五 编写代码，调试程序。

六、实验要求

根据归结原理编写程序（编程语言不限），要求给出如下过程：

1 求子句集：

(1) $\neg P(x) \vee Q(x)$

(2) $\neg P(y) \vee R(y)$

(3) $P(a)$

(4) $S(a)$

$$(5) \neg S(z) \vee \neg R(z) \quad (\neg G)$$

2 归结:

$$(6) R(a) \quad [(2), (3), \sigma_1 = \{a/y\}]$$

$$(7) \neg R(a) \quad [(4), (5), \sigma_2 = \{a/z\}]$$

$$(8) Nil \quad [(6), (7)]$$

实验七 决策树实验(西瓜分类) (*)

一、实验目的

掌握决策树分类学习方法，学习其中的ID3、C4.5和CART算法，并选用其中一个算法进行实验，同时懂得对数据文件进行预处理。

二、实验内容

使用决策树算法完成对西瓜数据集3.0的分类，根据西瓜的色泽、根蒂、敲声、纹理、脐部、触感、密度、含糖率共8个属性特征来判断西瓜是否是一个好瓜，西瓜数据集部分样例如下表所示。

编号	色泽	根蒂	敲声	纹理	脐部	触感	密度	含糖率	好瓜
1	青绿	蜷缩	浊响	清晰	凹陷	硬滑	0.697	0.460	是
2	乌黑	蜷缩	沉闷	清晰	凹陷	硬滑	0.774	0.376	是
3	乌黑	稍蜷	沉闷	稍糊	稍凹	硬滑	0.666	0.091	否

三、实验环境

推荐使用python语言，编译器推荐pycharm 2021.2.1，环境如下：

python=3.7.0

matplotlib=3.5.1

四、实验原理

决策树是一种描述对实例进行分类的树形结构。决策树由点和有向边组成。节点有两种类型：内部节点和叶节点。内部节点表示一种特征或者属性，叶节点表示一个分类。构建决策树时通常采用自上而下的方法，在每一步选择一个最好的属性来分裂。每次都找不同的切分点，将样本空间逐渐进行细分，最后把属于同一类的空间进行合并，就形成了决策边界，树的层次越深，决策边界的切分就越细，区分越准确，同时也越有可能产生过拟合。

决策树的建立开始，构建根节点，将所有训练数据放在根节点，选择一个最优特征，按照这一特征的取值将训练数据分割为子集，使各个子集有一个当前条件下最好的分类。如果这些子集能被基本正确分类，那么构造叶节点，将对应子集集中到叶节点。如果有子集不能被正确分类，那么就这些子集选择新的最优特征，继续对其进行分割，构建相应的节点。递归进行上述的操作，直到所有训练数据子集均能被正确分类。

信息熵(entropy) 熵是接收的每条消息中包含的信息的平均量，熵是对不确定性的测量。但是在信息世界，熵越高，则能传输越多的信息，熵越低，则意味着传输的信息越少。简单来说，熵表示随机变量的不确定性。

条件熵(conditional entropy) 在一个条件下，随机变量的不确定性。

信息增益(information gain) 信息增益=熵-条件熵。表示在一个条件下，信息不确定性减少的程度。

信息增益率(information gain ratio) 节点信息增益与节点分裂信息度量的比值。

ID3算法 ID3算法的核心是在决策树的各个结点上应用信息增益准则进行特征选择。

C4.5算法 相比ID3算法，C4.5算法更换了特征选择的标准，使用信息增益比进行特征选择。不直接选择增益率最大的候选划分属性，候选划分属性中找出信息增益高于平均水平的属性（这样保证了大部分好的的特征），再从中选择增益率最高的（又保证了不会出现编号特征这种极端的情况）对于连续值属性来说，可取值数目不再有限，因此可以采用离散化技术（如二分法）进行处理。将属性值从小到大排序，然后选择中间值作为分割点，数值比它小的点被划分到左子树，数值不小于它的点被分到右子树，计算分割的信息增益率，选择信息增益率最大的属性值进行分割。

CART算法 CART中用于选择变量的不纯度度量是Gini指数；如果目标变量是标称的，并且是具有两个以上的类别，则CART可能考虑将目标类别合并成两个超类别（双化）；如果目标变量是连续的，则CART算法找出一组基于树的回归方程来预测目标变量。CART是一棵二叉树，采用二元切分法，每次把数据切成两份，分别进入左子树、右子树。而且每个非叶子节点都有两个孩子，所以CART的叶子节点比非叶子多1。相比ID3和C4.5，CART应用要多一些，既可以用于分类也可以用于回归。

CART分类时，使用基尼指数（Gini）来选择最好的数据分割的特征，gini描述的是纯度，与信息熵的含义相似。CART中每一次迭代都会降低GINI系数。

剪枝 剪枝为了避免决策树“过拟合”样本，决策树的剪枝策略最基本的有两种：预剪枝（pre-pruning）和后剪枝（post-pruning）。

预剪枝（pre-pruning）：预剪枝就是在构造决策树的过程中，先对每个结点在划分前进行估计，若果当前结点的划分不能带来决策树模型泛华性能的提升，则不对当前结点进行划分并且将当前结点标记为叶结点。

后剪枝（post-pruning）：后剪枝就是先把整颗决策树构造完毕，然后自底向上的对非叶结点进行考察，若将该结点对应的子树换为叶结点能够带来泛华性能的提升，则把该子树替换为叶结点。

五、实验步骤

以C4.5决策树算法为例：

步骤一 将西瓜数据集划分为训练集和测试集，并将数据集中的每个属性值转换为唯一标签，如下图测试集转换结果所示(最后一列为分类标签，1表示是好瓜，0表示不是好瓜)。

```
0,0,0,1,1,1,5,5,1
2,0,0,2,1,1,6,6,1
2,0,0,1,1,0,7,7,1
2,0,1,2,1,0,8,8,0
0,2,2,1,0,1,9,9,0
1,2,2,0,0,0,15,11,0
1,1,0,0,0,1,10,14,0
```

步骤二 将数据集中的属性和属性值进行划分，核心代码如下所示：

```
1. def read_dataset(filename):
2.     fr = open(filename, 'r')
3.     all_lines = fr.readlines() # list 形式, 每行为1 个str
4.     # print all_lines
5.     labels = ['色泽', '根蒂', '敲声', '纹理', '脐部', '触感', '
        密度', '含糖率']
6.     dataset = []
7.     for line in all_lines[0:]:
8.         line = line.strip().split(',') # 以逗号为分割符拆分行
        表
9.         dataset.append(line)
10.    return dataset, labels
```

步骤三 使用C4.5算法来构造决策树，核心代码如下所示：

```

1.  def C45_createTree(dataset, labels, test_dataset):
2.      classList = [example[-1] for example in dataset]
3.      if classList.count(classList[0]) == len(classList):
4.          # 类别完全相同，停止划分
5.          return classList[0]
6.      if len(dataset[0]) == 1:
7.          # 遍历完所有特征时返回出现次数最多的
8.          return majorityCnt(classList)
9.      bestFeat = C45_chooseBestFeatureToSplit(dataset)
10.     bestFeatLabel = labels[bestFeat]
11.     print(u"此时最优索引为: " + (bestFeatLabel))
12.     print(u"最优索引对应的子集为:
13.         ", {bestFeatLabel: cds.ids2lables[bestFeatLabel]})
14.     C45Tree = {bestFeatLabel: {}}
15.     del (labels[bestFeat])
16.     # 得到列表包括节点所有的属性值
17.     featValues = [example[bestFeat] for example in dataset]
18.     uniqueVals = set(featValues)
19.     if pre_pruning:
20.         ans = []
21.         for index in range(len(test_dataset)):
22.             ans.append(test_dataset[index][-1])
23.         result_counter = Counter()
24.         for vec in dataset:
25.             result_counter[vec[-1]] += 1
26.         leaf_output = result_counter.most_common(1)[0][0]
27.         root_acc = cal_acc(test_output=[leaf_output] * len(test
28.             _dataset), label=ans)
29.         outputs = []
30.         ans = []
31.         for value in uniqueVals:
32.             cut_testset = splitdataset(test_dataset, bestFeat,
33.                 value)
34.             cut_dataset = splitdataset(dataset, bestFeat, value
35.                 )
36.             for vec in cut_testset:
37.                 ans.append(vec[-1])
38.             result_counter = Counter()
39.             for vec in cut_dataset:
40.                 result_counter[vec[-1]] += 1
41.             leaf_output = result_counter.most_common(1)[0][0]
42.             outputs += [leaf_output] * len(cut_testset)
43.             cut_acc = cal_acc(test_output=outputs, label=ans)
44.             if cut_acc <= root_acc:
45.                 return leaf_output
46.         for value in uniqueVals:
47.             subLabels = labels[: ]
48.             C45Tree[bestFeatLabel][value] = C45_createTree(
49.                 splitdataset(dataset, bestFeat, value),

```

```

46.         subLabels,
47.         splitdataset(test_dataset, bestFeat, value))
48.     if post_pruning:
49.         tree_output = classifytest(C45Tree, featLabels=['色泽', '根蒂', '敲声', '纹理', '脐部', '触感', '密度', '含糖率'], testDataSet=test_dataset)
50.         ans = []
51.         for vec in test_dataset:
52.             ans.append(vec[-1])
53.         root_acc = cal_acc(tree_output, ans)
54.         result_counter = Counter()
55.         for vec in dataset:
56.             result_counter[vec[-1]] += 1
57.         leaf_output = result_counter.most_common(1)[0][0]
58.         cut_acc = cal_acc([leaf_output] * len(test_dataset), ans)
59.         if cut_acc >= root_acc:
60.             return leaf_output
61.     return C45Tree
62. def C45_chooseBestFeatureToSplit(dataset):
63.     numFeatures = len(dataset[0]) - 1
64.     baseEnt = cal_entropy(dataset)
65.     bestInfoGain_ratio = 0.0
66.     bestFeature = -1
67.     for i in range(numFeatures): # 遍历所有特征
68.         featList = [example[i] for example in dataset]
69.         uniqueVals = set(featList) # 将特征列表创建成为 set 集合，元素不可重复。创建唯一的分类标签列表
70.         newEnt = 0.0
71.         IV = 0.0
72.         for value in uniqueVals: # 计算每种划分方式的信息熵
73.             subdataset = splitdataset(dataset, i, value)
74.             p = len(subdataset) / float(len(dataset))
75.             newEnt += p * cal_entropy(subdataset)
76.             IV = IV - p * log(p, 2)
77.         infoGain = baseEnt - newEnt
78.         if (IV == 0): # fix the overflow bug
79.             continue
80.         infoGain_ratio = infoGain / IV # 这个 feature 的 infoGain_ratio
81.         print(u"C4.5 中第%d 个特征的信息增益率为: %.3f" % (i, infoGain_ratio))
82.         if (infoGain_ratio > bestInfoGain_ratio): # 选择最大的 gain ratio
83.             bestInfoGain_ratio = infoGain_ratio
84.             bestFeature = i # 选择最大的 gain ratio 对应的 feature
85.     return bestFeature

```

步骤四 使用Matplotlib对剪枝后的决策树进行可视化展示，核心代码如下所

示：

```
1. def C45_Tree(inTree):
2.     fig = plt.figure(2, facecolor='white')
3.     fig.clf()
4.     axprops = dict(xticks=[], yticks=[])
5.     createPlot.ax1 = plt.subplot(111, frameon=False, **axprops)
6.     plotTree.totalw = float(getNumLeafs(inTree))
7.     plotTree.totalD = float(getTreeDepth(inTree))
8.     plotTree.xOff = -0.5 / plotTree.totalw
9.     plotTree.yOff = 1.0
10.    plotTree(inTree, (0.5, 1.0), '')
11.    plt.title("C4.5 决策树", fontsize=12, color='red')
12.    plt.show()
```

六、实验要求

1. 编写程序（编程语言不限），实现一个基于信息熵 (ID3、C4.5) 或者基于基尼指数 (CART) 进行划分选择的决策树算法。
2. 根据算法构造决策树，并将决策树进行可视化展示。

实验八 基于循环神经网络实现情感分类（*）

一、实验目的

熟悉和掌握循环神经网络的模型架构和工作原理，了解语言数据集的一些基本概念，例如词表、数据对齐等等，掌握对语言数据集的基本预处理操作，加深对分类问题的理解，并进一步熟悉使用paddle paddle进行深度学习。

二、实验内容

本实验使用IMDB数据集，该数据集中包括50000条偏向明显的评论，其中包括25000条训练数据和25000条测试数据，label为0（negative）和1（positive）。

本实验要求使用飞桨搭建循环神经网络模型，并利用上述数据集训练该模型；训练结束后保存模型，存储模型的参数；评估网络时，从参数文件中加载模型参数，再评估模型。

三、实验环境

1.建议使用百度提供的AI Studio平台做下面两个实验，该平台提供了免费的算力支持，并且已经集成了我们要用的paddle等库。

<https://aistudio.baidu.com/aistudio/index>

进入网页后→点击项目→创建项目→Notebook→选paddle 2.0.2版本→创建即可。
该实验建议选择gpu环境，否则训练十分缓慢。

2.如果要在自己电脑实验，Paddle Paddle的安装参考一下飞桨的官方文档：

<https://www.paddlepaddle.org.cn/install/quick>，paddlepaddle的版本建议选择2.0.2

四、实验原理

前馈神经网络只能单独地处理一个个的输入，每次的输入都是独立的，即网络的输出只依赖于当前的输入。但是在很多现实任务中，网络的输出不仅和当前实时的输入有关，也和过去一段时间的输入有关。此外，前馈神经网络难以处理时序数据，比如视频、语言、文本等。时序数据的长度一般是不固定的，而前馈神经网络要求输入和输出的维度都是固定的，不能任意改变。

因此，循环神经网络（RNN）是一类具有短期记忆能力的神经网络，一个简单的循环神经网络由输入层、隐藏层和输出层组成，公式如下：

$$s_t = f(Ux_t + Ws_{t-1} + b) \quad (4)$$

$$o_t = g(V_{s_t}) \quad (5)$$

其中，网络在 t 时刻接收到输入 x_t 后，得到的隐藏层的值为 s_t ，再经过一层线性层后，得到输出值 o_t 。并且， s_t 的值不仅取决于输入，还取决于 $t-1$ 时刻的隐藏层状态 s_{t-1} ， $f()$ 、 $g()$ 为激活函数， U 、 W 、 V 、 b 为模型参数。

五、实验步骤

步骤一 数据加载。

1. 由于IMDB是NLP领域常见的数据集，飞桨将其内置，路径为`paddle.text.datasets.Imdb`，并且通过`mode`参数可以控制加载训练集或测试集，数据集加载完毕后，需要从数据集中获得数据集的词表：`word_dict = train_dataset.word_idx`，并且需要在词表中添加特殊子符，用于对序列进行补充：`word_dict['<pad>'] = len(word_dict)`。

词表的结构为字典，key为单词，value为单词索引。

此外，还要定义模型的超参数：词表长度、单词嵌入长度、句子长度、批量大小、迭代轮次等等，核心代码如下：

```
1. # cpu/gpu 环境选择, 在paddle.set_device()输入对应运行设备
2. device = paddle.set_device('gpu')
3. # 加载训练数据
4. train_dataset = paddle.text.datasets.Imdb(mode='train')
5. # 加载测试数据
6. eval_dataset = paddle.text.datasets.Imdb(mode='test')
7.
8. # 获取数据集的词表 并且已经排序
9. word_dict = train_dataset.word_idx
10. # 在词表中添加特殊子符, 用于对序列进行补齐
11. word_dict['<pad>'] = len(word_dict)
12. # 参数设置
13. vocab_size = len(word_dict) + 1
14. emb_size = 256
15. seq_len = 200
16. batch_size = 32
17. epochs = 2
18. pad_id = word_dict['<pad>']
19. classes = ['negative', 'positive']
```

步骤二 数据处理。

用padding的方式对齐数据。在文本数据中，每一句话的长度都是不一样的，为了方便后续神经网络进行计算，常见的处理方式是将数据都统一成一样的长度。对

于较长的数据进行截断处理，对于较短的数据用特殊的符号（如：<pad>）填充。

用Dataset与DataLoader进行数据集加载，将前面准备好的训练集与测试集用Dataset与DataLoader封装后，完成数据的加载。Dataset的加载需要继承paddle.io.Dataset类，并重写__getitem__(), __len__()函数。DataLoader的加载则调用paddle.io.DataLoader()函数。

核心代码如下：

```

1.  # 读取数据归一化处理
2.  def create_padded_dataset(dataset):
3.      padded_sents = []
4.      labels = []
5.      for batch_ids, data in enumerate(dataset):
6.          sent, label = data[0], data[1]
7.          padded_sent = np.concatenate([sent[:seq_len], [pad_id]
8.          * (seq_len - len(sent))]).astype('int32')
9.          padded_sents.append(padded_sent)
10.         labels.append(label)
11.     return np.array(padded_sents), np.array(labels)
12.
13. # 对train, test 进行实例化
14. train_sents, train_labels = create_padded_dataset(train_dataset)
15. test_sents, test_labels = create_padded_dataset(eval_dataset)
16.
17. # 用Dataset 和DataLoader 进行数据集加载
18. class IMDBDataset(paddle.io.Dataset):
19.     '''
20.     继承 paddle.io.Dataset 类进行封装数据
21.     '''
22.     def __init__(self, sents, labels):
23.         self.sents = sents
24.         self.labels = labels
25.
26.     def __getitem__(self, index):
27.         data = self.sents[index]
28.         label = self.labels[index]
29.         return data, label
30.
31.     def __len__(self):
32.         return len(self.sents)
33.
34. train_dataset = IMDBDataset(train_sents, train_labels)
35. test_dataset = IMDBDataset(test_sents, test_labels)
36.
37. train_loader = paddle.io.DataLoader(train_dataset, return_list=True, shuffle=True, batch_size=batch_size, drop_last=True)
38. test_loader = paddle.io.DataLoader(test_dataset, return_list=True, shuffle=False, batch_size=batch_size, drop_last=True)

```

步骤三 模型配置。

在本实验中，将会使用一个序列特性的RNN网络进行文本编码，即 `paddle.nn.SimpleRNN(input_size, hidden_size, num_layers=1, activation='tanh', direction='forward', dropout=0, time_major=False, weight_ih_attr=None, weight_hh_attr=None, bias_ih_attr=None, bias_hh_attr=None)`。

该类是一个简单的循环神经网络，根据输入序列和给定的初始状态计算返回输出序列和最终状态，在该网络中的每一层对应输入的step，每个step根据当前时刻输入 $x(t)$ 和上一时刻状态 $h(t-1)$ 计算当前时刻输出 $y(t)$ 并更新状态 $h(t)$ ，该类接收参数如下：

`input_size`为输入层的神经元数；`hidden_size`为隐藏层的神经元数；`num_layers`为网络层数，默认为1；`direction`为网络迭代方向，可设置为forward或bidirect（或bidirectional）默认为forward；`time_major`指定input的第一维度是否为time steps，默认为False；`dropout`为dropout概率，指的是除第一层外每层输入时的dropout概率，默认为0；`activation`为网络中每个单元的激活函数，可以是`tanh()`或`ReLU`，默认为`tanh()`。

模型架构如下：

```

7.  class MyRNN(paddle.nn.Layer):
8.      #在初始化函数__init__中定义网络各层，重写forward方法，实现前向
      计算过程。
9.      def __init__(self):
10.         super(MyRNN, self).__init__()
11.         self.embedding = ..... # 嵌入层
12.         self.rnn = ..... # 循环神经网络
13.         self.linear = ..... # 线性层
14.         self.dropout = .....
15.
16.     # 前向计算
17.     def forward(self, inputs):
18.         .....
19.         return .....
```

步骤四 模型训练

定义好模型框架后：

1. 首先实例化模型model，在训练阶段，调用`model.train()`，开启训练模式，该模式具备反向传播功能。

2. 然后定义损失函数，在分类任务中，通常使用均方误差损失函数，

`paddle.nn.CrossEntropyLoss(reduction='mean')` 类提供了计算接口，参数 `reduction` 可以是 `'none'`、`'mean'`、`'sum'`，设为 `'none'` 时不适用约简，设为 `'mean'` 时返回 `loss` 的均值，设为 `'sum'` 时返回 `loss` 的和。

3. 定义优化器，神经网络模型的的反向传播需要定义传播规则，比如如何计算梯度，每个参数更新多少，更新的速度、幅度等，优化器正是用来实现上述目的的工具。`paddle.optimizer` 中实现了 **SGD**、**Adam** 等多种优化器。使用优化器也需要制定学习率，也就是梯度的更新步长，太大的学习率可能导致网络无法收敛，太小的学习率会使网络收敛很慢。本实验使用 `paddle.optimizer.Adam()` 进行优化。

4. 定义好模型、损失函数和优化器后，将数据分批次送入模型中后得到输出结果，根据输出结果和 `label` 计算 `loss`，然后执行梯度反向传播更新参数 (`loss.backward()`)，达到训练目的后，调用 `paddle.save()` 保存模型。后续进行预测时，只需要将训练好的模型参数加载到模型中即可。

默认迭代2轮 (`epochs=2`)，因为该数据集较大，可以自行调整。在模型训练过程中不要忘记输出训练损失值，该实验每500个批次输出一次，并且因为在后面模型评估时需要画图记录 `loss` 的变化过程，需要定义两个列表：

```
Iters, all_train_loss = [], []
```

在输出 `loss` 信息时，也需要将当前批次信息和损失信息加入列表中。

此外，在每个 `epoch` 后对模型进行一次评估，评估之前要将模型改为评估模式。大致图示如下，不需要一模一样，该模型较为简单，无法很好拟合该实验，所以 `loss` 值较大并且不下降。



训练模型的核心代码如下：

```

1. # 对模型进行封装
2. def train(model):
3. # 将模型设置为训练模式
4.     model.train()
5.     opt = paddle.optimizer.Adam(learning_rate=learning_rate, parameters=model.parameters())
6.     steps = 0
7.     Iters, total_loss, total_acc = [], [], []
8.     for epoch in range(epochs):
9.         for batch_id, data in enumerate(train_loader()):
10.             steps += 1
11.             sent, label = data[0], data[1]
12.             logits = model(sent)
13.             loss = paddle.nn.functional.cross_entropy(logits, label)
14.             acc = paddle.metric.accuracy(logits, label)
15.
16.             # 500 个 batch 输出一次结果
17.             if batch_id % 500 == 0:
18.                 Iters.append(steps)
19.                 total_loss.append(loss.numpy()[0])
20.                 total_acc.append(acc.numpy()[0])
21.                 print('epoch: {}, batch_id: {}, loss is: {}'.format(epoch, batch_id, loss.numpy()))
22.
23.             # 误差后向传播
24.             loss.backward()
25.             # 参数更新
26.             opt.step()
27.             opt.clear_grad()
28.             # print('batch: {} ing'.format(batch_id))
29.
30.         # 每个 epoch 后对模型进行评估
31.         model.eval()
32.         accuracies, losses = [], []
33.         for batch_id, data in enumerate(test_loader):
34.             sent, label = data[0], data[1]
35.             loss = paddle.nn.functional.cross_entropy(logits, label)
36.             acc = paddle.metric.accuracy(logits, label)
37.             accuracies.append(acc.numpy())
38.             losses.append(loss.numpy())
39.
40.         avg_acc, avg_loss = np.mean(accuracies), np.mean(losses)
41.         print('[validation]accuracy: {}, loss: {}'.format(avg_acc, avg_loss))
42.
43.     model.train()
44.
45. # 保存模型

```

```
46. paddle.save(model.state_dict(), str(epoch) + '_model_final.
    pdparams')
47.
48. # 可视化定义
49. def draw_process(title, color, iters, data, label):
50.     plt.title(title, fontsize=24)
51.     plt.xlabel('iter', fontsize=20)
52.     plt.ylabel(label, fontsize=20)
53.     plt.plot(iters, data, color=color, label=label)
54.     # 创建图例 图中的右上角
55.     plt.legend()
56.     # 显示网格线 1=True=默认显示; 0=False=不显示
57.     plt.grid()
58.     plt.show()
```

步骤五 模型评估与预测。

导入模型后，对模型进行评估，输出模型评估测试集时的平均损失值和精确度。并且使用该模型进行预测，在测试集中抽取一条数据输入到模型中，最后输出该数据的文本以及该文本的情感（positive或者negative）。

六、实验要求

1. 按照实验步骤编码完成数据的加载和预处理、模型的配置、模型的训练和评估，并绘制合适的图像描述模型的训练过程和模型的评估结果，绘制的图像合理即可。

2. 最终在测试集中抽取一条数据输入到模型中，最后输出该数据的文本以及该文本的情感（positive或者negative），结果如图所示：

[illegible]

附录A：实验报告封皮格式

实 验 报 告

课程名称 _____
题 目 _____
指导教师 _____
实验日期 _____
系 别 _____
专 业 _____
学生姓名 _____
班级/学号 _____
成 绩 _____

20xx年xx月xx日

附录B：实验报告格式

郑州大学计算机与人工智能学院实验报告

年级：_____ 专业：_____ 班级：_____

姓名：_____（电子签名） 学号：_____

指导老师：_____ 成绩：_____ 年____月____

日

实验题目：_____

一、 实验目的

二、 实验内容

三、 实验环境

四、 实验步骤

五、 实验结果与分析

六、 实验总结